Prompt Engineering / RAG / Agentic Workflow

Day 1 - AI for High-Value Food Processing & Operations

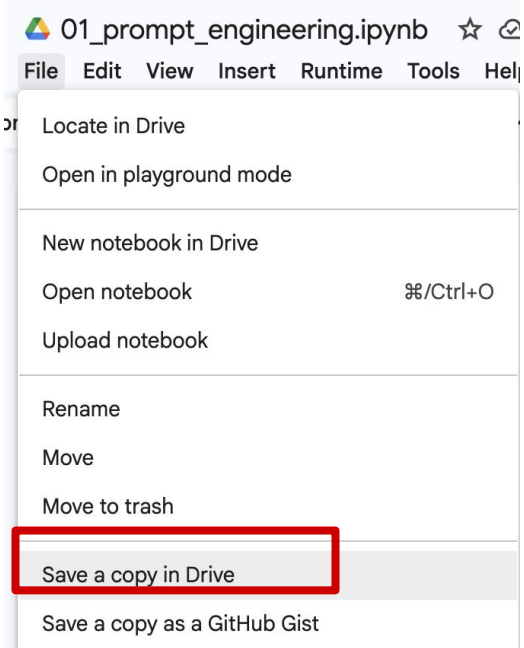
Agenda

1. **Prompt Engineering**
    a. What ? And Why ?
    b. System Prompt / User Prompt
    c. Best Practice 1: ICIO Prompting
    d. Best Practice 2: Improve Output Robustness
    e. Prompting Techniques
    f. Other LLM Configurations
    g. IRIS Prompting (Analysis / Validation Prompt)
2. **RAG**
    a. What ? and Why ?
    b. Pipeline
3. **Agentic Workflow**
    a. What ? And Why ?
    b. Concept : Node / Edge / State / Conditional Edge
4. **FAQ: Prompt Eng. / RAG on Production**

Prerequisites for all demos

- Demo Code : GitHub
  - *Once you open the file, please make a copy in your Google Drive first to avoid any errors.*
  - *You should run the notebook from your copied file.*

1. Prompt Engineering [`API-LangChain`]: Open in Colab
2. RAG [`API-LangChain`]: Open in Colab
3. Agentic Workflow [`API-LangChain-LangGraph`]: Open in Colab



Prerequisites for all demos (2)

- API Key
  1. **OpenAI API Key (Paid / Pay-as-you-go; closed-source OpenAI models)**
     i. Manual for Create an OpenAI API Key / Adding Credits / Dashboard (on GitHub):
  2. **Groq API Key (Free tier; open-source models)**
     i. Manual for Create a Groq API Key (on GitHub)

Once you have an API key, **keep it secret**.

Manual for Getting an API Key :

1. OpenAI API Key: Open in Google Docs
2. Groq API Key: Open in Google Docs

- Prerequisite: Basic Python Knowledge (e.g., variables, strings, lists, dictionaries, loops, functions, imports, and basic JSON handling)

Prerequisites for all demos (3)

- LangChain / LangGraph Framework:
  - If you are not familiar with these frameworks, this resource may be helpful (on GitHub):

Topic / Module	Presentation Slides
1. Prompt Engineering / RAG / Agentic Workflow	LangChain and LangGraph Prompt Engineering, RAG & Agentic Workflow

Prompt Engineering

Prompt Engineering - What ?

- Prompt engineering is the **process of designing, refining, and testing text inputs to guide generative AI models.**
- It is the critical "bridge" between human intent and machine output. By providing **clear context, specific instructions, and constraints**, you ensure the AI produces accurate, relevant, and high-quality responses.

Prompt Engineering - Why ?

- Because large language models (LLMs) like ChatGPT, Gemini, or Claude predict text based on how you prompt them, a **poorly phrased input usually yields a generic or incorrect answer**.
- **Effective prompt engineering eliminates guesswork**, **reduces hallucination,** and unlocks advanced AI capabilities.

Differences: System / User Prompt

Aspects	System Prompt	User Prompt
What ?	The foundational instructions that dictate an AI's behavior.	Specific instructions or questions a user provides to an AI.
Characteristic	**Fixed**, Definitions of General rules ,the main behavior of the AI/ LLM.	**Dynamic** based on user's query.
Scope	Global / Application-wide	Per Query / Turn-by-turn
Relevant Person	Developer / Owner	Users

Example: System / User Prompt

```
system_prompt_food_QA = """
# Role
- You are a QA support assistant for food manufacturing.
# General Rules
- You help organize evidence, identify signals, suggest hypotheses, and draft questions for human review.
- You must not decide product release, hold, quarantine, recall scope, CCP disposition, or deviation closure
- You must not invent lab results, timestamps, measurements, or regulatory conclusions.
- You must not introduce new pathogen names, legal standards, or regulatory conclusions unless they are supp
- If evidence is insufficient, state what is missing and say that QA/Food Safety approval is required.
- Keep responses concise unless the user explicitly asks for a detailed report.
""".strip()
```

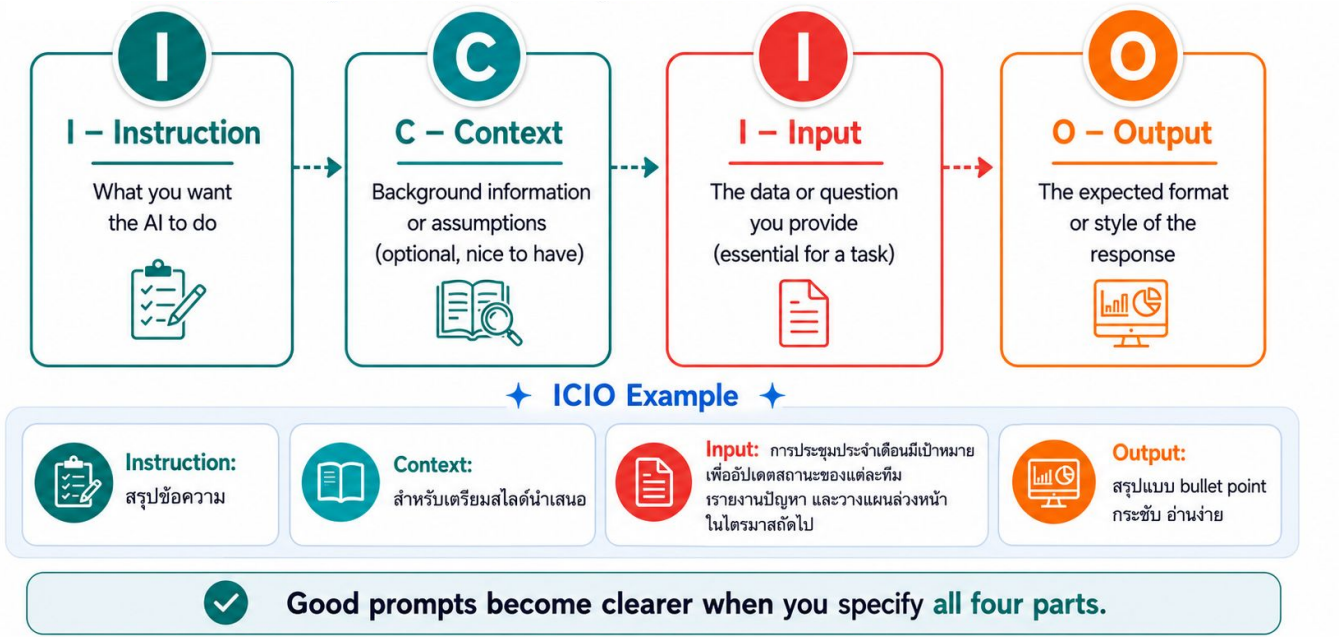
> System Prompt

```
user_naive_prompt = f"""
Analyze this food safety case and tell me what to do:

{case_paragraph}
"""

case_paragraph = """
Ready-to-eat chicken salad batch B-0519-N was produced on Line 2 during the night shift.
The process step under review is post-cook cooling. Cooling started at 00:00 with a recorded core temperatur
The procedure target is to cool from 60C to 21C within 2 hours, then to 5C within 4 additional hours.
At 02:00, the recorded core temperature was 24.8C. At 03:00, it was 14.2C. At 06:00, it was 5.7C.
The operator note says the batch was loaded into the chiller 20 minutes later than planned.
The chiller log shows one door-open alarm at 01:15, but the alarm duration is not included in the note.
QC observation says there was no visible packaging defect and the label and lot code were verified.
The previous three batches on the same line had no cooling deviation.
QA supervisor must review the case. Product release, hold, quarantine, recall scope, and CCP deviation dispo
""".strip()
```

> (Naive) User Prompt

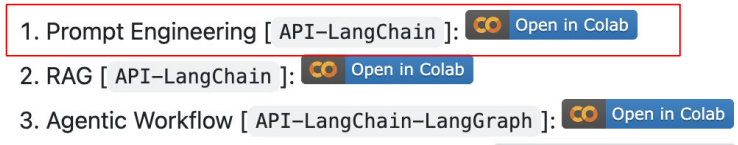
Best Practice 1: ICIO Prompting

Objective: To enhance the user's prompt by increasing its instructional and informational richness.



Demo 1: System / User Prompt

- Google Colab : 01_prompt_engineering.ipynb on GitHub
- **Remind: Once you open the file => Click `File` => `Save a copy in Drive` (skip this if you have done before.)**
- Prerequisite : API Key
  - For Free-tier : Get groq API key one (follow steps in the colab notebook)
  - For Paid-tier : Get OpenAI API key one (follow steps in the colab notebook)



Run: `01_prompt_engineering.ipynb → DEMO 1a-1b`
Observe: Without vs With System Prompt, Naive vs ICIO
Takeaway:
Role, context, constraints, and output format reduce ambiguity.

Demo 1: System / User Prompt

0. Setup

Choose the provider in cell 5 before running the notebook.

1. Run `pip install langchain-openai` if you have an OpenAI API key.

Otherwise, uncomment and run `pip install langchain-groq`.

```
# %%capture
!pip install -qU langchain-openai
```

9

```
# if using groq api (Free Tier) => don't
# !pip install -qU langchain-groq
```

2. PROVIDER = …

- **“openai” if you have an OpenAI API Key**
- **“groq” if you have a Groq API Key.**

```
import os
import json
import re
import time
import getpass

PROVIDER = "groq"  # CHANGE THIS LINE ONLY: openai or groq

if PROVIDER == "openai":
    from langchain_openai import ChatOpenAI
    os.environ["OPENAI_API_KEY"] = getpass.getpass("Enter your OpenAI API key: ")
    default_llm = ChatOpenAI(
        model="gpt-5-nano",
        temperature=0.6,
        max_retries=2,
    )
elif PROVIDER == "groq":
    from langchain_groq import ChatGroq
    os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq API key: ")
    default_llm = ChatGroq(
        model="openai/gpt-oss-120b",
        temperature=0.6,
        max_tokens=None,
        timeout=None,
        max_retries=2,
    )
else:
```

Demo 1: System / User Prompt

- System Prompt

```
simple_query_test = "I found a cooling deviation in a ready-to-eat product. What should I do?"
```

```
=== WITHOUT SYSTEM PROMPT ===
Thanks for asking. Here's a practical way to handle a cooling deviation for a rea

If you're a consumer (you found a cooling deviation on a ready-to-eat product you
- Do not eat the product. If the product is already opened, discard it.
- Do not try to "fix" or reheat it to make it safe. Some ready-to-eat foods can b
- Keep the packaging and receipt if possible, along with the lot/batch code and b
- Report the issue to the seller or manufacturer, using any recall/incident conta
- Check for any recall notices from the company or your local food safety authori
- If you feel unwell after consuming a product, seek medical advice.
```

```
# Role
- You are a QA support assistant for food manufacturing.
# General Rules
- You help organize evidence, identify signals, suggest hypotheses,
- You must not decide product release, hold, quarantine, recall sco
- You must not invent lab results, timestamps, measurements, or reg
```

```
=== WITH FOOD QA SYSTEM PROMPT ===
I can help outline the next steps. Final disposition (hold/recall) should come from QA/Food Safety. Here's a practical investigat

1) Immediate containment (per your site SOP)
- Segregate and quarantine the affected batch/units as directed by QA.
- Do not release any affected product until QA approves next actions.

2) Gather evidence and initial facts
- Product: code/lot, production date/time, POU/pack size, allergen info.
- Deviation description: exactly what cooling deviation occurred (e.g., failed to reach target temp within specified time, temper
- Equipment and process: cooling method (blast chiller, storage cooler, ambient), setpoints, equipment ID, any alarms or fault me
- Environment: line location, ambient temperature, airflow conditions.
- Quantity affected: number of units, packaging type, and whether any packaging shows defects.
- Operator actions: notes on how the line was run when deviation was observed.

3) Document process data
- Cooling logs or data from data loggers (target vs actual temps, times to reach key temps).
- Cooking step records (if relevant) and whether the product passed cooking before cooling.
- Any deviations in line speed, load size, or order of operations.

4) Initial risk assessment (without conclusions)
- Consider potential risk from temperature abuse during cooling for ready-to-eat product.
- Note any potential cross-contact or packaging integrity issues observed.
- Identify other lots or products that might be affected (same line, same batch, same cooling room, etc.).

5) Hypotheses to explore (no conclusions)
- Cooling profile not achieved due to equipment performance, improper loading, or timer mis-set.
- Cooling equipment under-dimensioned for batch size or placement caused slow cooling.
- Process steps out of sequence or missing cooling step.
- Temporary environmental factors (door open, high ambient temp).

6) Questions for QA review
- What is the critical limit for cooling for this product, and was it exceeded?
- Which CCPs and SOPs govern cooling, and were they followed?
- How many units/batches are implicated, and is there any evidence of other affected lots?
- What is the recommended disposition (hold, rework, discard) and communication plan?
```

```
# Role
- You are a travel planner who helps people organize

# General Rules
- You help with trip planning, route suggestions, boo
- You do not review food safety evidence or make QA d
- You keep answers practical and concise.
- If the user asks for food safety judgment, direct t
""".strip()
```

```
=== WITH SYSTEM PROMPT (TRAVEL PLANNER ROLE) ===
I can't judge safety. Please escalate the issue to your QA/Food Safety team right away.

- Do not consume the product. Quarantine it in a clearly marked, sealed container or ba
- Gather product details: brand, product name, size, lot/batch number, expiration date,
- Preserve evidence: keep the packaging, take photos of the label, lot code, and the de
- Document the incident: note who found it, where, and any symptoms or potential exposu
```

Demo 1: System / User Prompt

- User Prompt

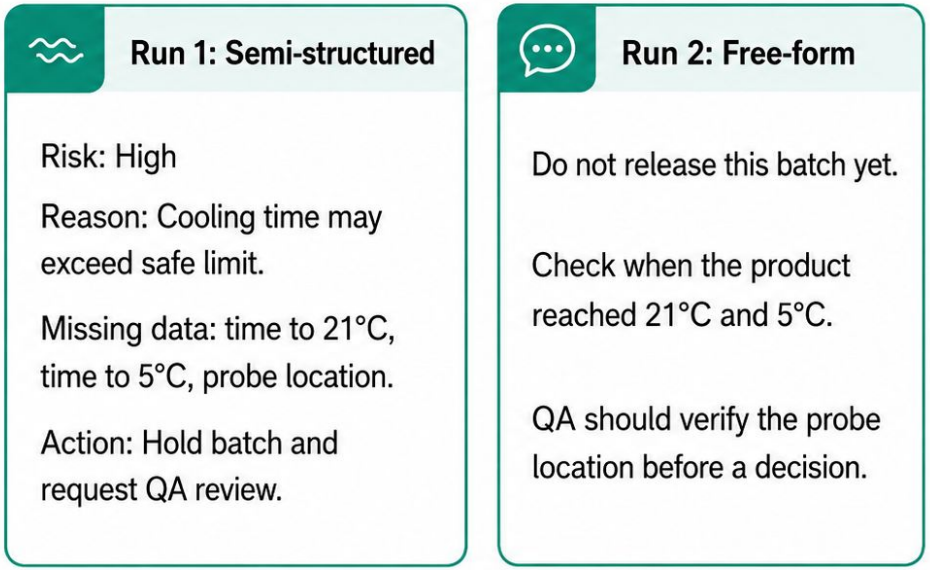
```
# cooling deviation
case_paragraph = """
Ready-to-eat chicken salad batch B-0519-N was produced on Line 2 during the night shift.
The process step under review is post-cook cooling. Cooling started at 00:00 with a recorded core temperature of 60.0C.
The procedure target is to cool from 60C to 21C within 2 hours, then to 5C within 4 additional hours.
At 02:00, the recorded core temperature was 24.8C. At 03:00, it was 14.2C. At 06:00, it was 5.7C.
The operator note says the batch was loaded into the chiller 20 minutes later than planned.
The chiller log shows one door-open alarm at 01:15, but the alarm duration is not included in the note.
QC observation says there was no visible packaging defect and the label and lot code were verified.
The previous three batches on the same line had no cooling deviation.
QA supervisor must review the case. Product release, hold, quarantine, recall scope, and CCP deviation disposition require Food Safety owner approval.
""".strip()
```

```
=== WITHOUT ICIO / WITHOUT CONSTRAINTS ===
Here's a concise QA-oriented analysis with the information needed to

1) What happened (short assessment)
- Cooling profile issue: The target is 60C → 21C within 2 hours, ther
- Observed data:
  - 00:00: 60.0C (start of cooling)
  - 02:00: 24.8C (above 21C; target not met by 2 h)
  - 03:00: 14.2C (below 21C)
  - 06:00: 5.7C (just above the 5C target)
- Delays/interruptions:
  - Operator note: batch loaded into chiller 20 minutes later than pl
  - Chiller log: one door-open alarm at 01:15; duration not provided.
- Other observations:
  - QC: no visible packaging defect; label/lot code verified.
  - Prior three batches on Line 2 had no cooling deviation.

Signals of concern
- Primary deviation: Did not meet the 21C target by 2 hours (02:00 re
- Secondary potential deviation: At 06:00 the temperature is 5.7C, sl
- Contributing factor: 20-minute late load into the chiller.
- Potential contributing factor: Door-open event at 01:15 (duration u

2) Key data gaps to close before disposition
- Exact duration of the 01:15 door-open alarm (and any additional doo
- Clarification of actual vs planned start time for chiller loading (
- Complete continuous temperature history from 00:00 through the end
- Chiller performance details: setpoints, compressor cycles, fan oper
- Any other line events during cooling (e.g., batch handling, agitato
- Confirmation of any subsequent product samples or sensory/micro ana

3) Hypotheses you can test with the missing data (human-in-the-loop)
- H1: The 2-hour target was missed primarily due to the 20-minute loa
- H2: The temperature drop after 02:00 indicates the cooling system e
```

```
=== WITH ICIO / WITH CONSTRAINTS ===
- Process
  - Post-cook cooling step for ready-to-eat chicken salac
  - Target: cool 60C to 21C within 2 hours; then 21C to 5
  - Observed: cooling started 00:00 at 60.0C; batch loade
  - Temperature readings: 02:00 = 24.8C; 03:00 = 14.2C; 0
  - QC note: no visible packaging defect; label and lot c
  - Historical context: previous three batches on Line 2

- Affected unit
  - Batch B-0519-N; Line 2; post-cook cooling step during

- Risk signal
  - Cooling target: 21C by 02:00 not met (02:00 reading 2
  - Cooling target: 5C by 06:00 not met (06:00 reading 5.
  - Operational deviation: 20-minute delay in loading int
  - Potential heat gain during cooling: door-open alarm a
  - Data gaps: duration of door-open event; full cooling
  - Packaging/label integrity not in question; historical

- Evidence / data sources
  - In-process temperature log: 00:00 (60.0C); 02:00 (24.
  - Chiller log: door-open alarm at 01:15 (duration not r
  - Operator note: 20-minute delay in loading into the ch
  - QC observation: no packaging defects; label and lot c
  - Line history: prior three batches on Line 2 with no c

- Decision owner
  - QA Supervisor (Case Reviewer). Note: final dispositic

- Missing context
  - Exact duration of the door-open event and its impact
  - Confirmation of the intended start time versus actual
  - Complete cooling profile between 00:00 and 03:00 (adc
  - Chiller performance data (compressor/fan status) and
  - Any corrective actions taken or planned (e.g., hold,
  - Confirmation of whether a less stringent interpretati
```

Best Practice 2: Improve Output Robustness

- **Asking the Model for Structured Output (e.g. JSON Format):** Instruct the model to return data in structured formats (e.g. JSON)



>> Without structured output, the response format may vary across runs.

This makes downstream tasks harder: recording, field extraction, validation, and QA action tracking.

In an automated pipeline, inconsistent format can cause parsing errors.

Best Practice 2: Improve Output Robustness

- **Asking the Model for Structured Output (e.g. JSON Format):** Instruct the model to return data in structured formats (e.g. JSON)

Structured JSON

```
{
  "risk_level": "high",
  "recommend_decision": "hold_batch",
  "missing_data": ["time_to_21c",
    "time_to_5c", "probe_location"],
  "action": "qa_review_required"
}
```

>> >> With structured output, the response follows a predictable format.

Each field can be parsed, validated, stored, and reused in QA workflows.

This makes the pipeline more robust and easier to trace.

Best Practice 2: Improve Output Robustness

- **Check Conditions: Tell the model what to do when required evidence is missing or when conditions are not met.**

Without Explicit Condition:

```
=== FREE-FORM MISSING VALUE OUTPUT ===
- batch_id: B-0519-N
- process_step: Post-cook cooling
- temperature_at_02_00_c: 24.8 C
- temperature_at_06_00_c: 5.7 C
- alarm_duration_minutes: Not provided (door-open alarm at 01:15 is noted, but duration is not stated)
- lab_result_summary: Not provided
- final_disposition_decision: Not provided (requires Food Safety owner approval for release/hold/quarantine/recall scope/CCP disposition)
- food_safety_owner_approval_status: Not provided (QA/Food Safety approval required)
- corrective_action_status: Not provided (no corrective action status documented)
```

With Explicit Condition: (return null if evidence is missing)

```
=== JSON null MISSING VALUE OUTPUT ===
{
  "boundary_condition_extract": {
    "batch_id": "B-0519-N",
    "process_step": "post-cook cooling",
    "temperature_at_02_00_c": 24.8,
    "temperature_at_06_00_c": 5.7,
    "alarm_duration_minutes": null,
    "lab_result_summary": null,
    "final_disposition_decision": null,
    "food_safety_owner_approval_status": "QA supervisor must review;
    "corrective_action_status": null
  }
}
```

Benefit:
Easier to parse because missing values are returned in a consistent format when the condition is met.

Demo 2: Improve Output Robustness

- Google Colab Link (Python Code Notebook): Same notebook

Run: `DEMO 2a–2c`
Observe: Free-form vs JSON output prompt, Check Condition (`null case`), Pydantic schema
Takeaway: Structured output makes results parseable, reusable, and traceable.

```
=== WITHOUT JSON / PLAIN TEXT OUTPUT ===
- data_source: Operator production note
  observed_fact: Batch B-0519-N loaded into chiller 20 minutes
  signal: Pre-cooling warm hold potentially affected cooling w
  possible_hypothesis: The 20-minute loading delay reduced effe
  supporting_evidence: Cooling started at 00:00 with 60.0C; ope
  missing_evidence: Exact planned load time; duration of loadi

- data_source: Chiller log
  observed_fact: door-open alarm at 01:15; alarm duration not p
  signal: Potential heat gain during door-open event impacting
  possible_hypothesis: Ingress of warm air or interruption of c
  supporting_evidence: 01:15 door-open alarm; 02:00 core temp 7
  missing_evidence: Alarm duration; whether door remained open;

- data_source: Temperature records
  observed_fact: 02:00 core temp 24.8C; 03:00 14.2C; 06:00 5.7(
  signal: Inconsistent cooling rate across the interval
  possible_hypothesis: Possible sensor calibration issue or mis
  supporting_evidence: Recorded temperatures at 02:00, 03:00, a
  missing_evidence: Probe calibration status; last calibration

- data_source: Process target documentation
  observed_fact: Target to cool 60C to 21C within 2 hours, ther
  signal: 2-hour target not met based on 02:00 measurement
  possible_hypothesis: Delay before reaching 21C or inaccurate
  supporting_evidence: Start temp 60.0C; 02:00 temp 24.8C
  missing_evidence: Exact start-of-cooling time relative to bat

- data_source: QC observation
  observed_fact: No visible packaging defect; label and lot coc
  signal: Packaging/label integrity unlikely contributor to coc
  possible_hypothesis: Deviation more likely process- or equipr
```

```
}
Use these fields:
data_source, observed_fact, signal, possibl

Case:
{case_paragraph}

Return valid JSON:
{{
  "diagnostic_signal_map": [
    {{
      "data_source": "string",
      "observed_fact": "string",
      "signal": "string",
      "possible_hypothesis": "string",
      "supporting_evidence": "string",
      "missing_evidence": "string"
    }}
  ]
}}
""".strip()
```

```
=== WITH STRUCTURED JSON ===
{
  "diagnostic_signal_map": [
    {
      "data_source": "Cooling process data (batch
      "observed_fact": "Cooling started at 00:00
      "signal": "Door-open event during cooling w
      "possible_hypothesis": "1) Door-open event
      "supporting_evidence": "01:15 door-open ala
      "missing_evidence": "Duration of the door-o
    },
    {
      "data_source": "Cooling process data (batch
      "observed_fact": "Operator reported the bat
      "signal": "Delayed chiller loading as a con
      "possible_hypothesis": "The 20-minute loadi
      "supporting_evidence": "20-minute delay not
      "missing_evidence": "Exact time when coolin
    }
  ]
}
```

Demo 2: Improve Output Robustness

- Google Colab Link (Python Code Notebook): Same notebook

Run: DEMO 2a–2c
Observe: Free-form vs JSON output prompt, **Check Condition (null case),** Pydantic schema
Takeaway: Structured output makes results parseable, reusable, and traceable.

```
Return valid JSON:
{{
  "boundary_condition_extract": {{
    "batch_id": "string or null",
    "process_step": "string or null",
    "temperature_at_02_00_c": "number or null",
    "temperature_at_06_00_c": "number or null",
    "alarm_duration_minutes": "number or null",
    "lab_result_summary": "string or null",
    "final_disposition_decision": "string or null",
    "food_safety_owner_approval_status": "string or null",
    "corrective_action_status": "string or null"
  }}
}}

Note:
If a field is not explicitly available in the case, return null.
""".strip()
```

```
=== JSON null MISSING VALUE OUTPUT ===
{
  "boundary_condition_extract": {
    "batch_id": "B-0519-N",
    "process_step": "post-cook cooling",
    "temperature_at_02_00_c": 24.8,
    "temperature_at_06_00_c": 5.7,
    "alarm_duration_minutes": null,
    "lab_result_summary": "QC observation: no visible packaging
    "final_disposition_decision": null,
    "food_safety_owner_approval_status": null,
    "corrective_action_status": null
  }
}

=== PARSED PYTHON VALUES ===
batch_id: 'B-0519-N' | python_type=str
process_step: 'post-cook cooling' | python_type=str
temperature_at_02_00_c: 24.8 | python_type=float
temperature_at_06_00_c: 5.7 | python_type=float
alarm_duration_minutes: None | python_type=NoneType
lab_result_summary: 'QC observation: no visible packaging defect
final_disposition_decision: None | python_type=NoneType
food_safety_owner_approval_status: None | python_type=NoneType
corrective_action_status: None | python_type=NoneType
```

Demo 2: Improve Output Robustness

- Google Colab Link (Python Code Notebook): Same notebook

Run: `DEMO 2a–2c`
Observe: Free-form vs JSON output prompt, **Check Condition (`null case`), Pydantic schema**
Takeaway: Structured output makes results parseable, reusable, and traceable.

```python
class BoundaryConditionExtract(BaseModel):
    batch_id: Optional[str] = Field(description="Batch ID")
    process_step: Optional[str] = Field(description="Process step if explicitly available")
    temperature_at_02_00_c: Optional[float] = Field(description="02:00 core temperature")
    temperature_at_06_00_c: Optional[float] = Field(description="06:00 core temperature")
    alarm_duration_minutes: Optional[float] = Field(description="Alarm duration")
    lab_result_summary: Optional[str] = Field(description="Lab result summary")
    final_disposition_decision: Optional[str] = Field(description="Final product disposition")
    food_safety_owner_approval_status: Optional[str] = Field(description="Approval status")
    corrective_action_status: Optional[str] = Field(description="Corrective action status")


class BoundaryConditionOutput(BaseModel):
    boundary_condition_extract: BoundaryConditionExtract
```

```python
structured_boundary_llm = default_llm.with_structured_output(BoundaryConditionOutput)

structured_boundary_result = structured_boundary_llm.invoke([
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": new_json_null_boundary_prompt}
])
```

```
1 structured_boundary_result.model_dump_json()
```

```
'{"boundary_condition_extract":{"batch_id":"B-0519-N","process_step":"post-cook cooling","temperature_at_02_00_c":24.8,"temperature_at_06_00_c":5.7,"alarm_duration_minutes":null,"lab_result_summary":null,"final_disposition_decision":null,"food_safety_owner_approval_status":null,"corrective_action_status":null}}'
```

Prompting Techniques

- **Chain-of-Thought (CoT) Prompting**
- **Few-shot Prompting**
- **Few-shot CoT**

Prompting Techniques: Chain-of-Thought

What is Chain of Thought (CoT) Prompting?

- **The Concept:** Chain of Thought (CoT) is a prompt engineering technique that significantly improves the ability of Large Language Models (LLMs) to **solve complex problems with greater accuracy and transparency**.
- **The Mechanism:** It prompts the LLM to "show its work," similar to how you would ask a human to explain their thought process.
- **The Approach:** Instead of jumping straight to a conclusion, it instructs the model to break down the problem into a series of intermediate, logical steps *before* generating the final answer.

Prompting Techniques: Chain-of-Thought

Even though modern models have a **built-in Reasoning Mode** or thought process, **writing a Prompt that forces the steps of thinking is still important in the following cases:**

1. **Controlling Domain-Specific Logic**: When we have industry-specific thought processes, questions, or requirements that the model must strictly follow.

2. **Ensuring Process Alignment**: Guaranteeing that the model truly follows our workflow or job topology, without skipping steps or making assumptions.

3. **Traceability for Review:** Being able to systematically trace the model's decision-making process to find where the results deviated.

Prompting Techniques: Chain-of-Thought

```
cot_prompt = f"""
Generate diagnostic questions for QA and Production.

Case:
{case_paragraph}

Diagnostic Signal Map:
{json.dumps(signal_map, ensure_ascii=False, indent=2)}

For each question, show a short structured reasoning trace using this path:
1. what_data_seen: the exact supplied data, record, or observation used
2. what_signal_observed: the pattern, deviation, anomaly, or weak signal noticed
3. hypothesis_to_test: a possible explanation to investigate, not a final root cause
4. evidence_needed: concrete record, measurement, interview, or log needed to test the hypothesis
5. decision_relevance: why this question helps QA/Production narrow the investigation
6. diagnostic_question: the actual question to ask humans

Constraints:
- Do not conclude final root cause.
- Do not recommend release, hold, quarantine, or recall.
- Use only supplied case data and the supplied signal map.
- Keep every field concise: 1 short sentence each.
- Each question must be record-checkable or follow-up-checkable.
- Each question must connect clearly to one signal or evidence gap.
- Do not add pathogen names, legal standards, or regulatory conclusions not supplied in the input.
- Return exactly 5 reasoning traces.
```

Guide the model with explicit "reasoning" steps.

```
Return valid JSON with exactly this schema:
{{
  "prompt_style": "cot_structured_trace",
  "question_reasoning_traces": [
    {{
      "question_id": "Q1",
      "target_role": "QA or Production or QA+Production",
      "what_data_seen": "string",
      "what_signal_observed": "string",
      "hypothesis_to_test": "string",
      "evidence_needed": "string",
      "decision_relevance": "string",
      "diagnostic_question": "string"
    }}
  ],
  "summary": "one short sentence describing how the trace improves auditability"
}}
""".strip()
```

> Includes intermediate reasoning traces for later validation.

> Suitable for safety and quality check tasks.

Prompting Techniques: Few-shot Prompting

Few-shot prompting is a technique where you provide the model with a few concrete examples within the prompt to guide its behavior toward your desired output.

Key Benefits:

- **Controls Style and Tone:** Ensures the model's response matches your specific voice, formality, or industry terminology.
- **Enforces Output Format:** Guarantees the data is structured exactly how you need it (e.g., specific templates, JSON, or lists).
- **Guides Logical Reasoning:** Demonstrates the exact thought process the model should follow to reach a conclusion (known as *Few-Shot Chain-of-Thought*).
- **Simplifies Complex Instructions: "Shows" rather than "tells."** Examples help bridge the gap when a task is too abstract or difficult to explain using explicit rules alone.

Prompting Techniques: Few-shot Prompting

Few-shot

```
few_shot_prompt = f"""
Example input:
A packaging line found three cartons from lot FG-204 with an older label artwork. The old label omitted one a

Example good narrative:
Observed facts: Three cartons from lot FG-204 were found with older label artwork before shipment.
Affected unit: Finished goods lot FG-204 on the packaging line.
Evidence reviewed: operator observation and QA label reconciliation start record.
Hypotheses under review: label roll-up mix-up or incomplete line clearance. These are hypotheses only.
Missing evidence: full label reconciliation, line clearance record, count of affected units.
Approval required: QA/Food Safety must approve label disposition, rework, release, or customer notification.

Now draft a deviation narrative for the cooling deviation case using the same structure.
```

Output

```
Observed facts: Ready-to-eat chicken salad batch **B-0519-N** was produced on **Line 2** (night shift).

Affected unit: **Batch B-0519-N** cooling step (post-cook cooling) on **Line 2**.

Evidence reviewed: Recorded core temperature values at **00:00, 02:00, 03:00, and 06:00**; operator not

Hypotheses under review (not conclusions): **(1)** The **2-hour-to-21°C** target was not met due to **a

Missing evidence: **Planned vs actual** chiller loading timestamps and the procedure definition of when

Approval required: **QA/Food Safety owner must review** the case. Food Safety owner approval is require
```

Few-shot Prompting Example: Output style followed the given example.

Prompting Techniques: Few-shot CoT

- **Guides Logical Reasoning:** Demonstrates the exact thought process the model should follow to reach a conclusion (known as *Few-Shot Chain-of-Thought*).

```
few_shot_cot_prompt = f"""
You are helping QA and Production generate diagnostic questions for a food safety deviation investigation.

Follow the reasoning trace style shown in the examples.

Example 1

Input:
A packaging line found three cartons from lot FG-204 with older label artwork.
The old label omitted one allergen statement.
Shipment had not started.
QA label reconciliation was opened.

Reasoning Trace Example:
Step 1: I see that three cartons from lot FG-204 used older label artwork before shipment.
Step 2: This suggests a label version mismatch affecting finished goods.
Step 3: A hypothesis to test is label roll-up mix-up or incomplete line clearance.
Step 4: Evidence still missing includes the full label reconciliation result, line clearance record, and total affected carton count.
Step 5: Therefore, the first diagnostic question should ask QA to confirm label reconciliation, line clearance, and affected quantity.

Example 2

Input:
```

Demo 3: Prompting Techniques

- Google Colab Link (Python Code Notebook): Same notebook

Run: DEMO 3a–3c
Observe: No CoT vs structured trace, Zero-shot vs Few-shot, Few-shot CoT
Takeaway: Instructions guide reasoning; examples guide style and reasoning pattern.

```
Generate diagnostic questions for QA and Production.

Case:
{case_paragraph}

Diagnostic Signal Map:
{json.dumps(signal_map, ensure_ascii=False, indent=2)}

For each question, show a short structured reasoning trace using this path:
1. what_data_seen: the exact supplied data, record, or observation used
2. what_signal_observed: the pattern, deviation, anomaly, or weak signal noticed
3. hypothesis_to_test: a possible explanation to investigate, not a final root cause
4. evidence_needed: concrete record, measurement, interview, or log needed to test the hypothesis
5. decision_relevance: why this question helps QA/Production narrow the investigation
6. diagnostic_question: the actual question to ask humans
```


```
{
  "prompt_style": "cot_structured_trace",
  "question_reasoning_traces": [
    {
      "question_id": "Q1",
      "target_role": "QA+Production",
      "what_data_seen": "Cooling process data shows a door-open alarm at 01:15 with no duration provided.",
      "what_signal_observed": "Door-open event suggesting heat ingress may have slowed initial cooling.",
      "hypothesis_to_test": "The door-open duration could have caused heat ingress slowing cooling, affecting reaching 2
      "evidence_needed": "Exact duration of the 01:15 door-open alarm and any additional door openings during 00:00–02:0
      "decision_relevance": "Clarifies if a door breach affected cooling rate, guiding corrective actions.",
      "diagnostic_question": "What was the exact duration of the 01:15 door-open alarm, and were there any other door op
    },
    {
      "question_id": "Q2",
      "target_role": "QA+Production",
      "what_data_seen": "Operator note: batch loaded into the chiller 20 minutes later than planned; 02:00 temp 24.8C; 0
      "what_signal_observed": "Delay in loading into chiller reduces effective cooling time.",
      "hypothesis_to_test": "The 20-minute loading delay contributed to not meeting the 2h target.",
      "evidence_needed": "Exact actual loading time and scheduled vs actual times; chiller performance data for 00:00–02
      "decision_relevance": "Assesses impact of operational schedule deviation on cooling performance.",
      "diagnostic_question": "What was the exact actual time the batch was loaded into the chiller, and what was the pla
    },
    {
      "question_id": "Q3",
      "target_role": "QA+Production",
      "what_data_seen": "Thermal trajectory shows 60.0C at 00:00; 02:00 24.8C; 03:00 14.2C; 06:00 5.7C; 21C not met by 2
      "what_signal_observed": "Initial cooling slower than target; potential equipment or load factors.",
      "hypothesis_to_test": "Chiller performance or batch load characteristics limited cooling rate.",
      "evidence_needed": "Chiller setpoints, airflow, compressor status during 00:00–02:00; batch weight/volume; ambient
      "decision_relevance": "Identify whether equipment or product factors contributed to slow cooling.",
      "diagnostic_question": "What were the chiller setpoints and status (compressor/airflow) during 00:00–02:00, and wh
```

Other LLM Configurations (1)

- **Temperature Setting** (**Legacy Model** e.g. GPT-4o):

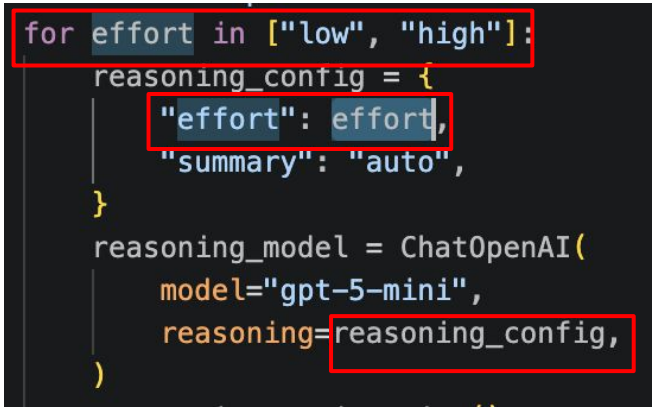
  What it does: Controls the randomness and creativity of the model's responses.

  - **Low Temperature** (e.g., 0.1 – 0.3): Makes the output **highly deterministic,** focused, and precise. Ideal for factual queries, math, and code generation.
  - **High Temperature (**e.g., 0.7 – 1.0): Increases **variety and unpredictability. Ideal for brainstorming, creative writing, and open-ended roleplay.**

```
PROVIDER = "openai"  # openai or groq

if PROVIDER == "openai":
    from langchain_openai import ChatOpenAI
    os.environ["OPENAI_API_KEY"] = getpass.getpass("Enter your OpenAI API key: ")
    default_llm = ChatOpenAI(
        model="gpt-5-nano",
        temperature=0.6,
        max_retries=2,
    )
```

Other LLM Configurations (2)

- **Reasoning Effort Setting** (**Modern Models** e.g. GPT-5):
  What it does: Dictates how much internal thinking time and compute the model allocates to a problem before it starts writing the response.
  - **Low/Medium Effort:** Minimizes latency for straightforward questions, giving quick, direct answers.
  - **High Effort:** Allows the model to deeply analyze complex problems, generate internal chains of thought, self-correct errors, and execute advanced logic or multi-step coding tasks.



Demo 4: Other LLM Configurations

Run: DEMO 4a–4b
Observe: Temperature 0 vs 1, reasoning effort low vs high
Takeaway: Temperature affects variation; reasoning effort affects evidence checking.

```python
for temp in [0, 1]:
    if PROVIDER == "openai":
        temp_llm = ChatOpenAI(
            model="gpt-4o-mini",
            temperature=temp,
            max_retries=2,
        )
```

```
Write 3 alternative internal email subject lines for the QA team about this cooling deviation.

Case:
{case_paragraph}

Diagnostic Signal Map:
{json.dumps(signal_map, ensure_ascii=False, indent=2)}

Instructions:
- Preserve the facts from the case.
- Vary wording, tone, and structure.
- Each subject line should be short and concise.
- Do not ask diagnostic questions.
- Do not judge product disposition.
- Return a concise numbered list only.
""".strip()
```

```
================================================================================
TEMPERATURE = 0 | 3 RUNS
================================================================================

--- Round 1 ---
1. Review Required: Cooling Deviation for Batch B-0519-N on Line 2
2. Attention Needed: Post-Cook Cooling Issues for Chicken Salad Batch B-0519-N
3. Urgent: Cooling Timeline Deviation Analysis for Batch B-0519-N

--- Round 2 ---
1. Review Required: Cooling Deviation for Batch B-0519-N on Line 2
2. Attention Needed: Post-Cook Cooling Issues for Chicken Salad Batch B-0519-N
3. Urgent: Cooling Timeline Deviation Analysis for Batch B-0519-N

--- Round 3 ---
1. Review Required: Cooling Deviation for Batch B-0519-N on Line 2
2. Attention Needed: Post-Cook Cooling Issues for Chicken Salad Batch B-0519-N
3. Urgent: Cooling Timeline Deviation Analysis for Batch B-0519-N
```

```
================================================================================
TEMPERATURE = 1 | 3 RUNS
================================================================================

--- Round 1 ---
1. Review Needed: Cooling Deviation for Batch B-0519-N on Line 2
2. Urgent: Analysis of Cooling Delay for Night Shift Chicken Salad Batch
3. Action Required: Deviation in Cooling Process for Batch B-0519-N

--- Round 2 ---
1. "Review of Cooling Deviation for Batch B-0519-N on Line 2"
2. "Post-Cook Cooling Assessment: Batch B-0519-N Departure from Target"
3. "Cooling Process Deviation Case: Batch B-0519-N Requires QA Supervisor Review"

--- Round 3 ---
1. "Cooling Deviation Review Required for Batch B-0519-N"
2. "Post-Cook Cooling Deviation Analysis: Batch B-0519-N"
3. "Request for QA Review: Cooling Process for Batch B-0519-N"
```

Demo 4: Other LLM Configurations

Run: DEMO 4a–4b
Observe: Temperature 0 vs 1, **reasoning effort low vs high**
Takeaway: Temperature affects variation; reasoning effort affects evidence checking.

```python
for effort in ["low", "high"]:
    reasoning_config = {
        "effort": effort,
        "summary": "auto",
    }
    reasoning_model = ChatOpenAI(
        model="gpt-5-mini",
        reasoning=reasoning_config,
    )
```

```
reasoning_effort_prompt = f"""
You are verifying statements about a food manufacturing deviation case.

Case:
{case_paragraph}

Diagnostic Signal Map:
{json.dumps(signal_map, ensure_ascii=False, indent=2)}
```

```
# cooling deviation
case_paragraph = """
Ready-to-eat chicken salad batch B-0519-N was produced on Line 2 during the night shift.
The process step under review is post-cook cooling. Cooling started at 00:00 with a recorded core temperature of 60.0C.
The procedure target is to cool from 60C to 21C within 2 hours, then to 5C within 4 additional hours.
At 02:00, the recorded core temperature was 24.8C. At 03:00, it was 14.2C. At 06:00, it was 5.7C.
The operator note says the batch was loaded into the chiller 20 minutes later than planned.
The chiller log shows one door-open alarm at 01:15, but the alarm duration is not included in the note.
QC observation says there was no visible packaging defect and the label and lot code were verified.
The previous three batches on the same line had no cooling deviation.
QA supervisor must review the case. Product release, hold, quarantine, recall scope, and CCP deviation disposition requi
""".strip()
```

```
signal_map = {
  "diagnostic_signal_map": [
    {
      "data_source": "Cooling process data (batch B-0519-N, Line 2, night shift): coo
      "observed_fact": "Cooling started at 00:00 with core temperature 60.0C. Chiller
      "signal": "Door-open event during cooling with schedule delay contributing to c
      "possible_hypothesis": "1) Door-open event at 01:15 caused heat ingress, slowin
      "supporting_evidence": "01:15 door-open alarm; 02:00 temp 24.8C (above 21C); 03
      "missing_evidence": "Duration of the door-open alarm; exact start time of cooli
    },
    {
      "data_source": "Cooling process data (batch B-0519-N) and operator/line history
      "observed_fact": "Operator reported the batch was loaded into the chiller 20 mi
      "signal": "Delayed chiller loading as a contributor to cooling deviation.",
      "possible_hypothesis": "The 20-minute loading delay reduced effective cooling t
      "supporting_evidence": "20-minute delay noted; cooling temps indicate slower in
      "missing_evidence": "Exact time when cooling began relative to loading; batch w
    }
  ]
}
```

```
Statements:
1. Batch B-0519-N was loaded into the chiller 20 minutes later than planned.
2. The chiller door-open alarm duration at 01:15 is known.
3. The 02:00 core temperature is above the 21C target.
4. The same calibrated probe location was used for all recorded temperatures.
5. QA/Food Safety approval is required before any disposition decision.
```

Demo 4: Other LLM Configurations

Run: DEMO 4a–4b
Observe: Temperature 0 vs 1, **reasoning effort low vs high**
Takeaway: Temperature affects variation; reasoning effort affects evidence checking.

```
================================================================================
REASONING EFFORT = "low"
================================================================================
1)
- verdict: PASS
- evidence: "The operator note says the batch was loaded into the chiller 20 minutes later than planned."
- reason: Operator note explicitly states a 20-minute loading delay.
- missing_data: N/A

2)
- verdict: FAIL
- evidence: "The chiller log shows one door-open alarm at 01:15, but the alarm duration is not included in the note."
- reason: Duration is explicitly not provided in the available records.
- missing_data: N/A

3)
- verdict: PASS
- evidence: "At 02:00, the recorded core temperature was 24.8C" and target is "reach 21C within 2 hours."
- reason: 24.8°C at 02:00 is above the 21°C target.
- missing_data: N/A

4)
- verdict: UNKNOWN (ยังสรุปไม่ได้)
- evidence: No information provided about probe location consistency or calibration across readings.
- reason: Case data does not state whether the same calibrated probe location was used for all measurements.
- missing_data: Probe location records, calibration logs, or measurement procedure documentation.

5)
- verdict: PASS
- evidence: "QA supervisor must review the case. Product release, hold, quarantine, recall scope, and CCP deviation di
- reason: The case explicitly requires QA review and Food Safety owner approval for disposition decisions.
- missing_data: N/A

Elapsed time: 9.77 seconds
```

```
REASONING EFFORT = “high”
5)
- verdict: UNKNOWN (ยังสรุปไม่ได้)
- evidence: "QA supervisor must review the
case" and "Product release, hold, quarantine,
recall scope, and CCP deviation disposition
require Food Safety owner approval."
- reason: ยังสรุปไม่ได้ – Food Safety approval is
explicitly required, but it is not stated
that QA approval (as sign-off) is required
before disposition (only that QA must
review).
- missing_data: Clarification or documented
procedure stating whether QA approval (not
just review) is required and the required
sign-off workflow for disposition decisions.

Elapsed time: 68.10 seconds
```

Take-home: Analysis Prompt Template (IRIS)

Prompt สำหรับให้ Artificial Intelligence ช่วยวิเคราะห์ข้อมูลและตั้งสมมติฐาน

ใส่ข้อความนี้ใน Prompt

คุณคือผู้ช่วยวิเคราะห์ข้อมูลในอุตสาหกรรมอาหาร
บริบท: [อธิบายกระบวนการและปัญหาที่เลือก]
ข้อมูลที่มี: [ระบุ data source เช่น batch record, lab result, sensor data, complaint data]

งานที่ต้องการให้วิเคราะห์:
1. ระบุ signal ที่สำคัญ
2. ตั้งสมมติฐานที่เป็นไปได้
3. ระบุหลักฐานที่สนับสนุนแต่ละสมมติฐาน
4. ระบุหลักฐานที่ยังขาด
5. เสนอคำถามที่ควรถามทีม Production หรือ Quality Assurance เพิ่ม

ข้อจำกัดที่ต้องใส่

- ห้ามสรุป Root cause เกินหลักฐาน
- ห้ามเสนอ Release, quarantine หรือ Recall โดยไม่มี Human approval

Question: สามารถพัฒนา Prompt Template นี้ได้อย่างไรบ้าง จากหลักการของ ICIO Prompting and Best Practice ?
(Solution: Prompt ข้างต้นมี Instruction, Context, Input Block แล้ว และมี Check Condition Block
โดยอาจยังขาดการกำหนด Output Format Structure ที่ชัดเจน ซึ่งอาจทำให้ LLM ตอบไม่ชัดเจน และยากต่อการตรวจสอบ)

Take-home: Validation Prompt Template (IRIS)

Prompt สำหรับตรวจคำตอบก่อนนำไปใช้จริงในโรงงานอาหาร

ตรวจ 5 เรื่อง

- คำตอบอ้างอิงข้อมูลจริงหรือไม่
- ย้อนกลับไปที่ lot, batch, line, shift หรือเวลาได้ไหม
- มีสมมติฐานใดที่ยังไม่มีหลักฐานสนับสนุน
- มีความเสี่ยงด้าน food safety, compliance หรือ customer impact หรือไม่
- จุดใดต้องให้มนุษย์อนุมัติก่อนดำเนินการ

หากหลักฐานไม่พอ ให้ระบุว่า “ยังสรุปไม่ได้” และบอกว่าต้องใช้ข้อมูลอะไรเพิ่ม

Question: สามารถพัฒนา Prompt Template นี้ได้อย่างไรบ้าง จากหลักการของ ICIO Prompting and Best Practice ?
(Solution: Prompt มี Instruction, และมีการ Check Condition(s)
โดยยังขาด Input Block, Context Block, และการกำหนด Output Structure เพื่อให้ใช้ผลการตรวจต่อได้ง่าย)

RAG

Retrieval / Augmented / Generation: Grounding LLMs with Vectorized Knowledge

RAG: What ? and Why ?

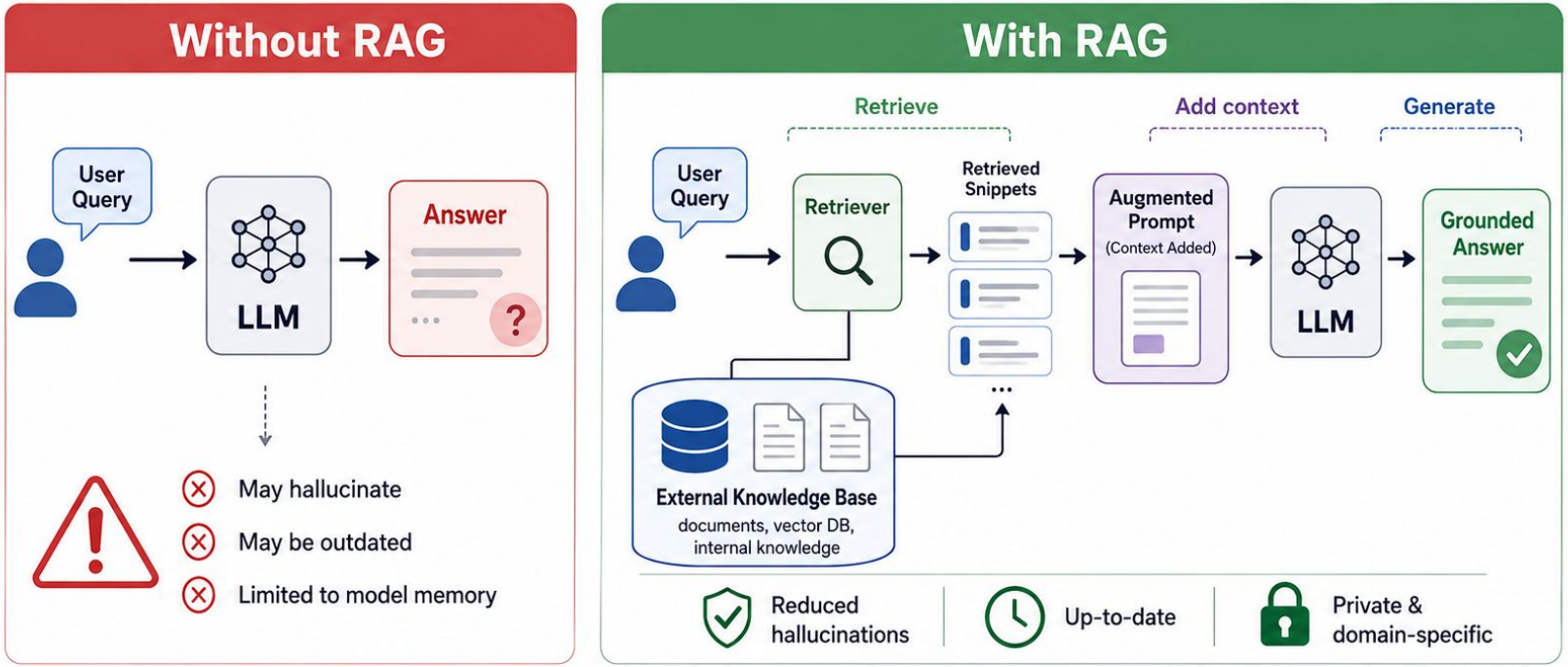
What is RAG? A process that combines the "linguistic capabilities of an LLM" with "external databases."

- **R - Retrieval:** Fetching relevant information related to the user's query from a database (e.g., Vector Database).
- **A - Augmented:** Appending the retrieved information as "context" directly into the Prompt.
- **G - Generation:** Having the LLM synthesize the final answer strictly based on the provided context.

Why RAG?

- **Reduce Hallucinations:** Forces the AI to answer using verifiable facts or references, rather than relying solely on the model's internal memory.
- **Always Up-to-date:** Allows you to instantly ingest new files or update information in the system without the time and expense of fine-tuning a new model.
- **Private & Domain-Specific:** Enables the AI to answer in-depth questions about internal organizational documents or confidential information that isn't available on the internet.

RAG: What ? and Why ?



RAG = **Retrieval** (Find relevant information) + **Augmentation** (Add context to the prompt) + **Generation** (Generate informed answer)

The RAG Pipeline Overview

1. Store

Transform knowledge base into searchable vectors.

- Raw Documents
- Chunking
- Embedding
- Vector Database

2. Retrieve

Find relevant chunks based on semantic meaning.

- User Query
- Query Embedding
- Similarity Search
- Get Top-n Context

3. Augment

Construct a highly contextual prompt.

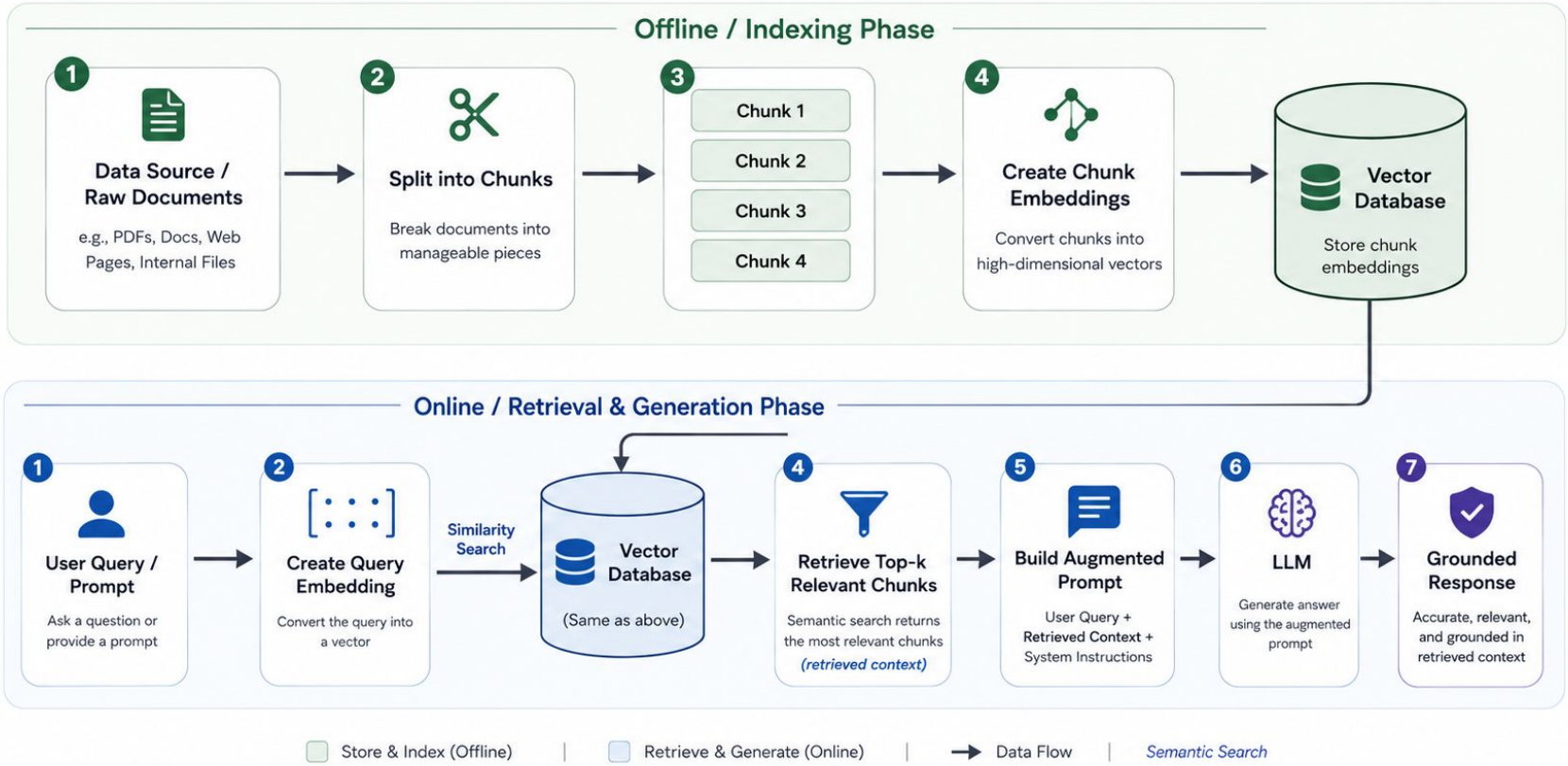
- Inject Top-n into Prompt
- Combine with User Query
- System Instructions

4. Generate

LLM reasoning based on retrieved evidence.

- Process Context
- Synthesize Answer
- Output Grounded Response

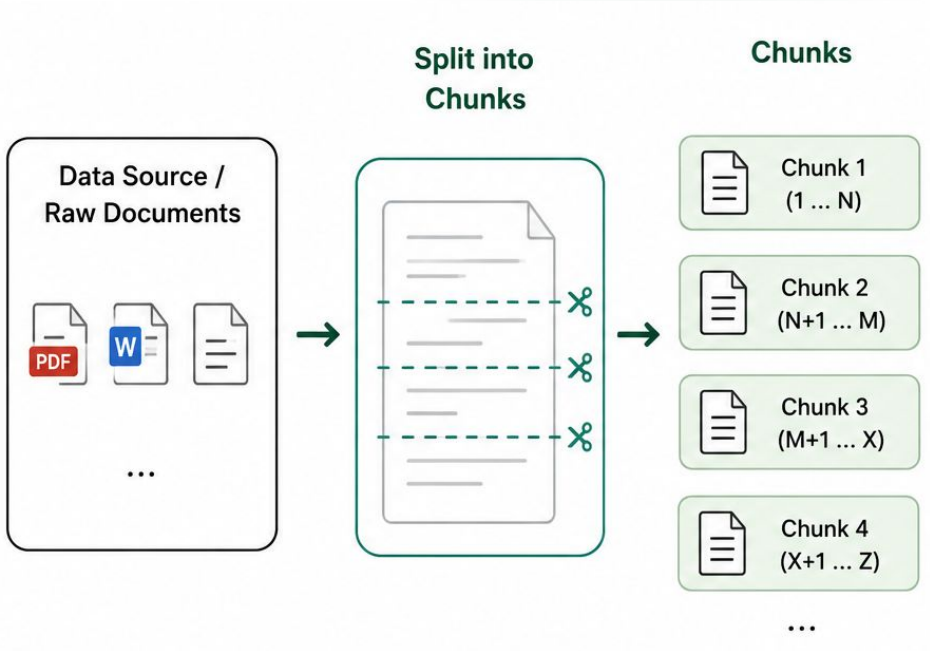
The RAG Pipeline



Chunking: Managing Context Boundaries

Chunking is the process of splitting large documents into smaller, manageable pieces to optimize **Retrieval Precision**.

- **Fixed-Size:** Splitting by a strict token count (e.g., 512 tokens).
- **Recursive:** Adapting to structural boundaries (paragraphs, sentences) to preserve context.
- **Overlap:** Including tokens from the previous chunk to ensure no context is lost at the borders.

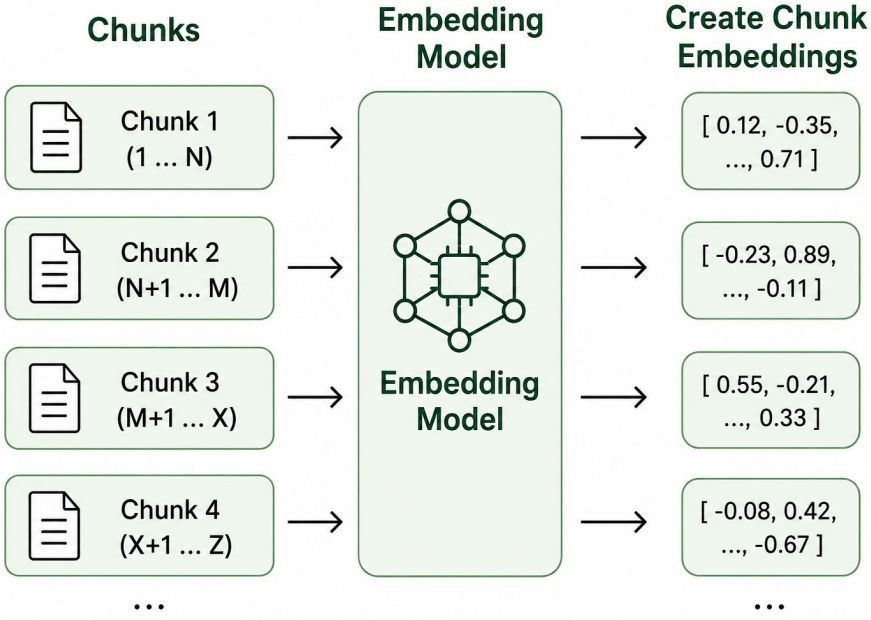


Embedding: Text to Vector Space

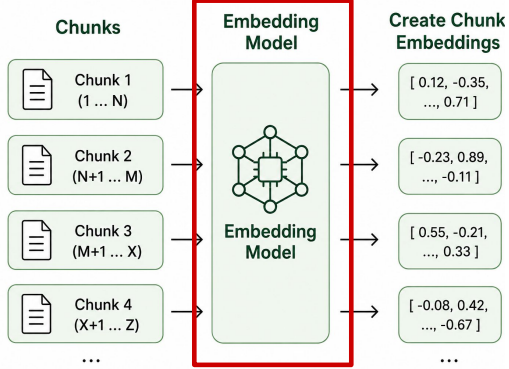
Bridging Language to Mathematics

Embeddings convert semantic meaning into high-dimensional numerical coordinates, allowing mathematical comparison of concepts.

- **Semantic Proximity:** Words with similar meanings (e.g., "King" and "Queen") are grouped closely in the vector space.
- **Fixed Dimensions:** Every text fragment becomes an array of fixed length (e.g., d = 1536).
- **Dense Representation:** Each value represents abstract features learned by the model.



Embedding: Embedding Models



BGE-M3

THE MULTILINGUAL GENERALIST

A heavyweight model designed for robustness across 100+ languages and long-context documents (8k tokens).

Best For: Complex, global data.
Feature: Dense & Sparse retrieval.
Trade-off: Higher compute demand.
Language: 100+ languages (including Thai)

all-miniLM-L6

THE EFFICIENT SPEEDSTER

An ultra-lightweight transformer optimized for extreme speed and low-latency production environments.

Best For: Real-time APIs, mobile.
Feature: 14,000+ sentences/sec.
Trade-off: Smaller context (384 tokens).
Language: English

```
from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
```

Code Example: HuggingFace Embedding Models

Numerical Anatomy of a Vector

1024

Common Dimensions (d)

(Example: bge-m3)

A Vector is a fixed-size array of floating-point numbers representing semantic features.

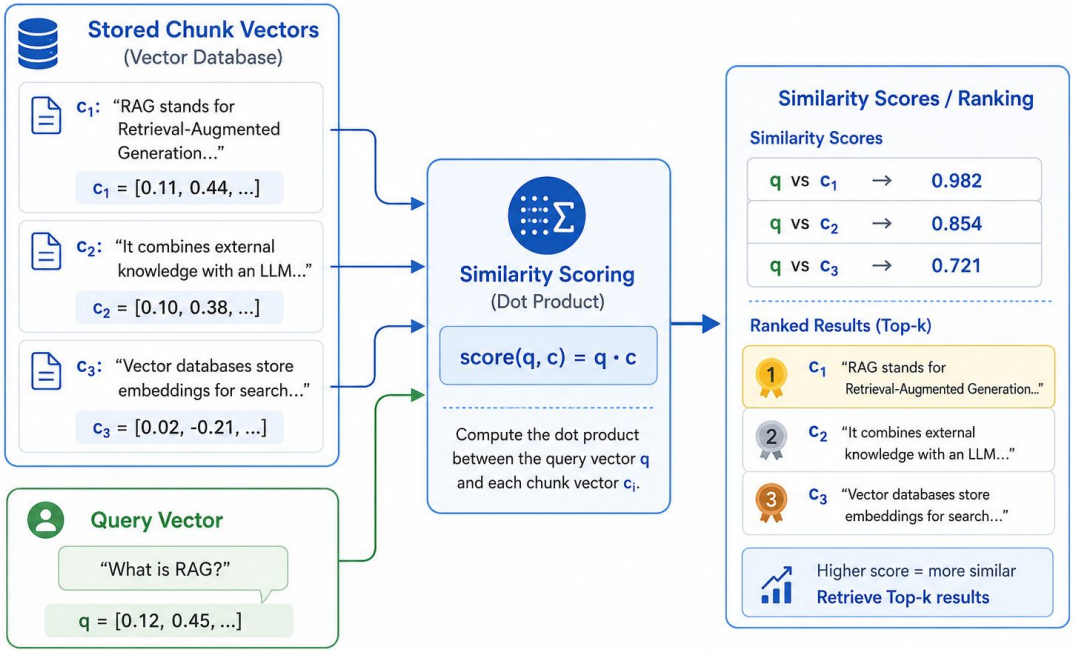
$$= \begin{pmatrix} 0.0142 \\ -0.4519 \\ \vdots \\ 0.8234 \end{pmatrix}$$

Similarity Search: Dot Product

To find relevance, we calculate the mathematical proximity between the **Query Vector (q)** and the **Chunk Vectors (c)** stored in the database.

$$\text{Sim Score} = q \cdot c = \sum_{i=1}^{d} q_i c_i$$

This calculates the sum of the products of corresponding entries, measuring the alignment of two vectors in space.



Ranking & Selecting Top-n Context

Query Text: "What is RAG?"

Query Vector (q): [0.12, 0.45, -0.11, ..., 0.87]

Rank	Text Fragment (Chunk)	Chunk Vector	Sim Score
#1	"RAG stands for Retrieval-Augmented Generation..."	[0.11, 0.44, -0.09, ..., 0.87]	0.982
#2	"It combines external search tools with LLMs..."	[0.10, 0.38, 0.02, ..., 0.75]	0.854
#3	"Vector databases are used to store embeddings..."	[0.02, -0.21, 0.15, ..., 0.40]	0.721

Augmented Generation

Augment

Inject the retrieved **Top-n results** into the prompt as "Context".

```
System: Answer using this context:

{Rank#1 Text}
{Rank#2 Text}

User: {Original Query}
```

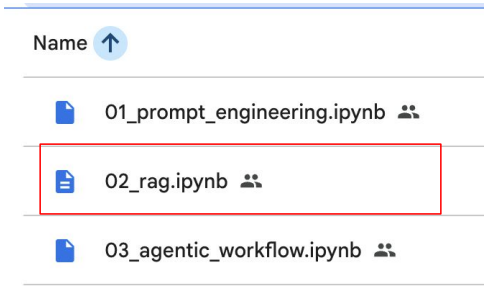
Generate

The LLM processes the augmented prompt and synthesizes the final output.

- **Grounded:** The answer is rooted in the provided context.
- **Accurate:** Significantly minimizes hallucinations.
- **Traceable:** Responses can be linked back to the source chunks.

Demo: RAG

- Google Colab : code
- **Remind: Once you open the file => Click `File` => `Save a copy in Drive` (skip this if you have done before.)**
- Prerequisite : API Key
  - For Free-tier : Get groq API key one (follow steps in the colab notebook)
  - For Paid-tier : Get OpenAI API key one (follow steps in the colab notebook)



Demo: RAG

0. Setup

Choose the provider in cell 5 before running the notebook.

1. Run `pip install langchain-openai` if you have an OpenAI API key.

Otherwise, uncomment and run `pip install langchain-groq`.

```
# %%capture
!pip install -qU langchain-openai
```

9

```
# if using groq api (Free Tier) => don't
# !pip install -qU langchain-groq
```

2. PROVIDER = …

- **“openai” if you have an OpenAI API Key**
- **“groq” if you have a Groq API Key.**

```
import os
import json
import re
import time
import getpass

PROVIDER = "groq"  # CHANGE THIS LINE ONLY: openai or groq

if PROVIDER == "openai":
    from langchain_openai import ChatOpenAI
    os.environ["OPENAI_API_KEY"] = getpass.getpass("Enter your OpenAI API key: ")
    default_llm = ChatOpenAI(
        model="gpt-5-nano",
        temperature=0.6,
        max_retries=2,
    )
elif PROVIDER == "groq":
    from langchain_groq import ChatGroq
    os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq API key: ")
    default_llm = ChatGroq(
        model="openai/gpt-oss-120b",
        temperature=0.6,
        max_tokens=None,
        timeout=None,
        max_retries=2,
    )
else:
```

Demo : RAG

Demo Documents (downloaded automatically when the code in the notebook is executed)

PDF

มาตรฐานสินค้าเกษตร

มกษ. 9035-2563

THAI AGRICULTURAL STANDARD

TAS 9035-2020

การปฏิบัติที่ดีสำหรับโรงคัดบรรจุผักและผลไม้สด

GOOD MANUFACTURING PRACTICES FOR

PACKING HOUSE OF FRESH FRUITS AND VEGETABLES

สำนักงานมาตรฐานสินค้าเกษตรและอาหารแห่งชาติ

กระทรวงเกษตรและสหกรณ์

มาตรฐานสินค้าเกษตร

มกษ. 9039-2556

THAI AGRICULTURAL STANDARD

TAS 9039-2013

การปฏิบัติที่ดีสำหรับการผลิตผักและผลไม้สดตัดแต่งพร้อมบริโภค

GOOD MANUFACTURING PRACTICES FOR

READY-TO-EAT FRESH PRE-CUT FRUITS AND

VEGETABLES

สำนักงานมาตรฐานสินค้าเกษตรและอาหารแห่งชาติ

กระทรวงเกษตรและสหกรณ์

ICS 67.020 ISBN

Demo : RAG

1. Prompt Engineering [`API-LangChain`]: Open in Colab
2. RAG [`API-LangChain`]: Open in Colab
3. Agentic Workflow [`API-LangChain-LangGraph`]: Open in Colab

Notebook: `02_rag.ipynb`

Stack Used:

- **LangChain Framework**
- **Embedding Model : `bge-m3`**
- **Vector Database : `ChromaDB`**

Observe:
Load docs → chunk → embed → retrieve Top-n → generate grounded answer

Takeaway: RAG grounds answers with retrieved evidence.

```
pdf_sources = [
    {
        "doc_id": "tas_9039_fresh_cut_rte",
        "title": "TAS 9039-2556: Good Manufacturing Practices for Ready-to-Eat Fresh Pre-Cut Fruits and Veg
        "url": "https://tas2go.acfs.go.th/upload_standard/43_th.pdf",
        "filename": "tas_9039_fresh_cut_rte.pdf",
        "topic": "fresh-cut ready-to-eat produce",
    },
    {
        "doc_id": "tas_9035_packing_house",
        "title": "TAS 9035-2563: Good Practices for Fresh Fruit and Vegetable Packing House",
        "url": "https://tas2go.acfs.go.th/upload_standard/412_th.pdf",
        "filename": "tas_9035_packing_house.pdf",
        "topic": "fresh produce packing house",
    },
    {
        "doc_id": "acfs_decree_248_china_export",
        "title": "ACFS Manual: Food Producer Registration for Export to China under Decree 248",
        "url": "https://e-book.acfs.go.th/backend/uploads/Download/8cf05fc81b78906756467afbd81974c6.pdf",
        "filename": "acfs_decree_248_china_export.pdf",
        "topic": "China export registration",
    },
]
```

rag_pdfs
- acfs_decree_248_china_export.pdf
- tas_9035_packing_house.pdf
- tas_9039_fresh_cut_rte.pdf

```
from langchain_community.document_loaders import PyPDFLoader

pages = []

for item in pdf_sources:
    path = PDF_DIR / item["filename"]
    loader = PyPDFLoader(str(path))
    loaded_pages = loader.load()

    for page in loaded_pages:
        page.metadata.update({
            "doc_id": item["doc_id"],
            "title": item["title"],
            "url": item["url"],
            "topic": item["topic"],
            "source_path": str(path),
        })
```

Demo : RAG

Notebook: `02_rag.ipynb`

Stack Used:

- **LangChain Framework**
- **Embedding Model : `bge-m3`**
- **Vector Database : `ChromaDB`**

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=900,
    chunk_overlap=180,
    add_start_index=True,
)

splits = text_splitter.split_documents(pages)
```

```python
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings

# bge-m3 works better for multilingual Thai/English retrieval, b
embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")

vector_store = Chroma(
    collection_name="food_safety_specific_pdf_rag_demo",
    embedding_function=embeddings,
)

document_ids = vector_store.add_documents(documents=splits)
print(f"Stored {len(document_ids)} chunks in vector store")
```

Observe:
Load docs → chunk → embed → retrieve Top-n → generate grounded answer

Takeaway: RAG grounds answers with retrieved evidence.

```
Query key: china_decree_248_registration
Query: ภายใต้ China Decree 248 อาหารประเภทใดต้องขึ้นทะเบียนผ่านหน่วยงานกำกับดูแลของประเทศผู้ส่งออก และอาหารประเภทใดสามารถขึ้นทะเบียนด้วยตน

Retrieved Documents
================================================================================
[Source 1] ACFS Manual: Food Producer Registration for Export to China under Decree 248
URL: https://e-book.acfs.go.th/backend/uploads/Download/8cf95fc81b78906756467afbd81974c6.pdf
Page: 9 | chunk: 112
• Fresh and dehydrated vegetables, dried beans • Natural plant spices • Nuts and seeds • Dried fruits • Unroasted coffee
--------------------------------------------------------------------------------
[Source 2] ACFS Manual: Food Producer Registration for Export to China under Decree 248
URL: https://e-book.acfs.go.th/backend/uploads/Download/8cf95fc81b78906756467afbd81974c6.pdf
Page: 46 | chunk: 173
The General Administration of Customs of China (GACC). Regulations of the People's Republic of China on the Registration
--------------------------------------------------------------------------------
[Source 3] ACFS Manual: Food Producer Registration for Export to China under Decree 248
URL: https://e-book.acfs.go.th/backend/uploads/Download/8cf95fc81b78906756467afbd81974c6.pdf
Page: 4 | chunk: 101
1. ระเบียบ 248 (Decree 248) คืออะไร Page | 4 สำนักงานมาตรฐานสินค้าเกษตรและอาหารแห่งชาติ www.acfs.go.th 1. ระเบียบ 248 (Decree 248
--------------------------------------------------------------------------------
[Source 4] ACFS Manual: Food Producer Registration for Export to China under Decree 248
URL: https://e-book.acfs.go.th/backend/uploads/Download/8cf95fc81b78906756467afbd81974c6.pdf
Page: 5 | chunk: 103
2. ภาพรวมของการขึ้นทะเบียนผู้ผลิตอาหารตามระเบียบ 248 (Decree 248) Page | 5 สำนักงานมาตรฐานสินค้าเกษตรและอาหารแห่งชาติ www.acfs.go.th
--------------------------------------------------------------------------------
[Source 5] ACFS Manual: Food Producer Registration for Export to China under Decree 248
URL: https://e-book.acfs.go.th/backend/uploads/Download/8cf95fc81b78906756467afbd81974c6.pdf
Page: 42 | chunk: 165
คำถามที่พบบ่อย (Frequently Asked Questions, FAQ) Page | 42 สำนักงานมาตรฐานสินค้าเกษตรและอาหารแห่งชาติ www.acfs.go.th โดยข้อมูลที่ถูกระ
--------------------------------------------------------------------------------
```

Demo : RAG

Notebook: `02_rag.ipynb`

Observe:
Load docs → chunk → embed → retrieve Top-n → generate grounded answer

Takeaway: RAG grounds answers with retrieved evidence.

```
=== WITHOUT RAG ===
ข้อความที่ให้มาไม่ชัดเจนเกี่ยวกับประเภทของอาหารที่ต้องขึ้นทะเบียนผ่านหน่วยงานกำกับดูแลของประเทศผู้ส่งออก และประเภทของอาหารที่สามารถขึ้นทะเบียนด้วยตนเองผ่าน CIFER ได้

อย่างไรก็ตาม ตามข้อมูลทั่วไปเกี่ยวกับ China Decree 248 (Decree of the General Administration of Customs of the People's Republic of China No. 24

อาหารประเภทที่ต้องขึ้นทะเบียนผ่านหน่วยงานกำกับดูแลของประเทศผู้ส่งออก ได้แก่

- อาหารที่มีส่วนผสมของสัตว์หรือส่วนผสมที่เกี่ยวข้องกับสัตว์
- อาหารที่มีส่วนผสมของพืชหรือส่วนผสมที่เกี่ยวข้องกับพืช
- อาหารที่มีส่วนผสมของวัตถุอื่นๆ ที่อาจมีผลกระทบต่อสุขภาพ

อาหารประเภทที่สามารถขึ้นทะเบียนด้วยตนเองผ่าน CIFER ได้ ได้แก่

- อาหารที่ไม่มีส่วนผสมของสัตว์หรือส่วนผสมที่เกี่ยวข้องกับสัตว์
- อาหารที่ไม่มีส่วนผสมของพืชหรือส่วนผสมที่เกี่ยวข้องกับพืช
- อาหารที่ไม่มีส่วนผสมของวัตถุอื่นๆ ที่อาจมีผลกระทบต่อสุขภาพ

อย่างไรก็ตาม ข้อมูลข้างต้นอาจไม่ครอบคลุมทั้งหมด และควรตรวจสอบข้อมูลล่าสุดและเป็นทางการจากหน่วยงานกำกับดูแลของประเทศจีนเพื่อให้ได้ข้อมูลที่ถูกต้องและสมบูรณ์.
```

```
================================================================================
WITH RAG
================================================================================

ตามเอกสาร [ACFS Decree 248 Manual, p. 10] อาหารประเภทที่ต้องขึ้นทะเบียนผ่านหน่วยงานกำกับดูแลของประเทศผู้ส่งออก ได้แก่

- ผลิตภัณฑ์จากพืช (Fresh and dehydrated vegetables, dried beans)
- น้ำมันพืช (Natural plant spices)
- เมล็ดพืช (Nuts and seeds)
- ผลไม้แห้ง (Dried fruits)
- กาแฟและกะทิ (Unroasted coffee and cocoa beans)
- อาหารที่มีคุณสมบัติพิเศษ (Special Dietary Foods, Excluding milk-based infant formula)
- อาหารที่มีคุณสมบัติทางโภชนาการ (Functional foods)

อาหารประเภทเหล่านี้ต้องขึ้นทะเบียนผ่านหน่วยงานกำกับดูแลของประเทศผู้ส่งออกก่อนที่จะสามารถส่งออกไปยังจีนได้

ในทางกลับกัน อาหารประเภทที่สามารถขึ้นทะเบียนด้วยตนเองผ่าน CIFER ได้แก่ สินค้าอาหารและเครื่องดื่มอื่นๆ ที่ไม่รวมอยู่ในรายการที่กำหนดไว้ข้างต้น [ACFS
Decree 248 Manual, p. 10]
```

Agentic Workflow

What ? and Why ?

What ? : An Agentic Workflow is a structured AI workflow where an LLM does not only generate a one-time answer, but works through multiple connected steps to complete a task.

Why ? : To handle tasks that require multiple checks, decisions, or human approval beyond a single prompt. It makes the process more controllable, traceable, and suitable for real-world workflows.

Example: From Single Prompt to Multi-step Workflow

Instead of:

```
User Prompt → LLM → Final Answer
```

we design a workflow such as:

```
Input → Evidence Check → Risk Analysis → Action Recommendation → Human / Logic Decision → Final Log
```

Agentic Workflow Outline

- **Concept 1: State**
- **Concept 2: Nodes**
- **Concept 3: Edges**
- **Concept 4: Conditional Edges + Human-in-the-loop**

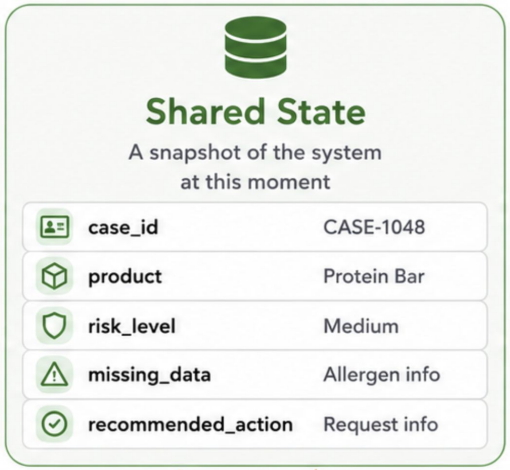
Concept 1: State

1. State (The Evolving Memory)

- The State is the backbone of the workflow; it is a shared, globally **accessible data structure that holds the snapshot of the entire system at any given moment.**
- It acts as both the **short-term working memory** and the evolving context across the entire session. As the workflow progresses, nodes read this state and return updates to it.

Example of Our Data Table (State)
Agents and functions (will be called "Nodes" in the next slide) can "read" and "update."

```python
class FoodSafetyState(TypedDict, total=False
    case_id: str
    product: str
    batch_id: str
    cooling_minutes_to_5c: int
    max_temperature_c: float
    lab_result_status: str
    batch_record_complete: bool
    sensor_record_complete: bool
    complaint_count_14d: int

    operator_note: str
    sensor_trend_note: str
    qa_note: str
    complaint_note: str
```

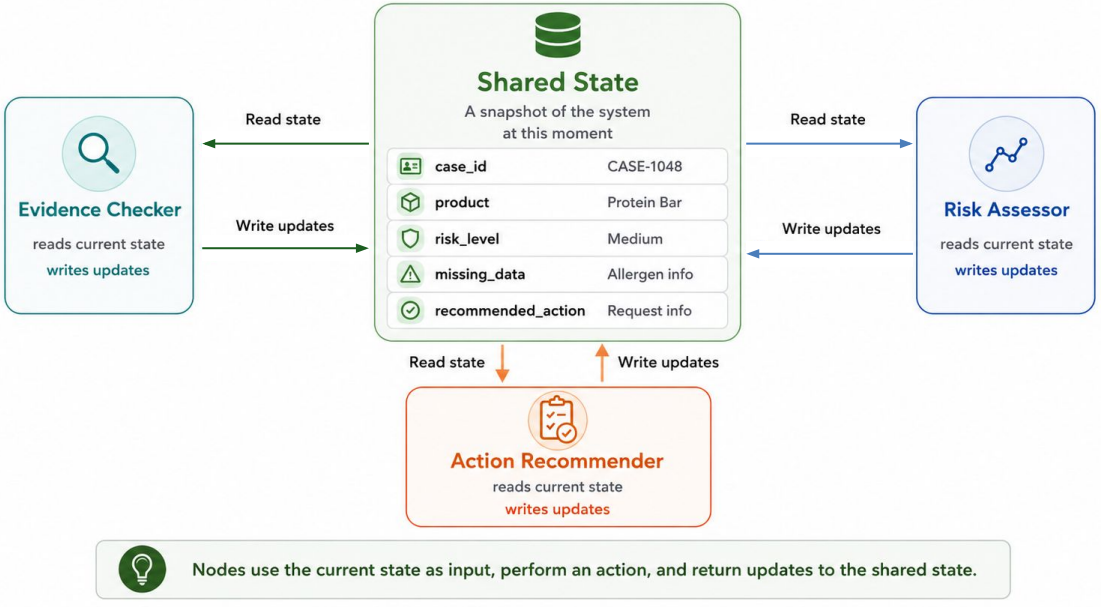


Concept 2: Nodes

2. Nodes (The Actionable Logic)

- Nodes represent the **individual steps, agents, or tools in your workflow.**

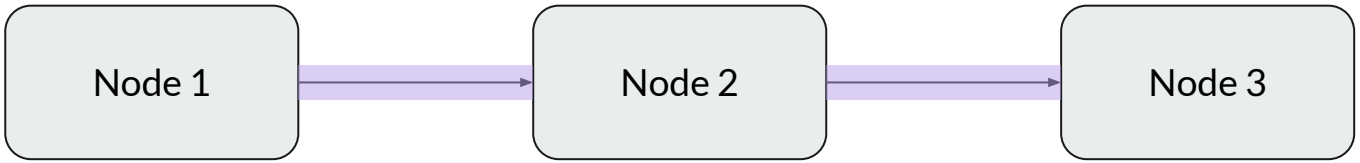
- They are simply callable functions or classes that take the current State as an input, perform an operation **(e.g., call an LLM, run a Python script, search the web),** and **output updates back to the State.**



Concept 3: Edges

3. Edges (The Flow)

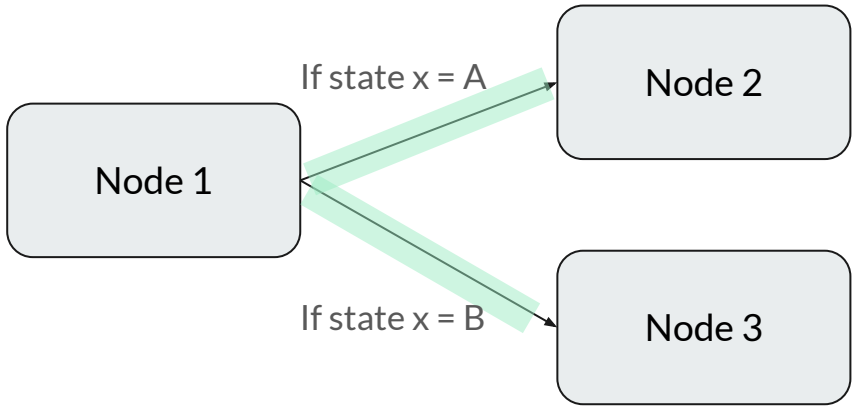
- Edges define the sequence and connections between your nodes.
- They establish the rules of traversal, dictating exactly which node or nodes the workflow should execute next.



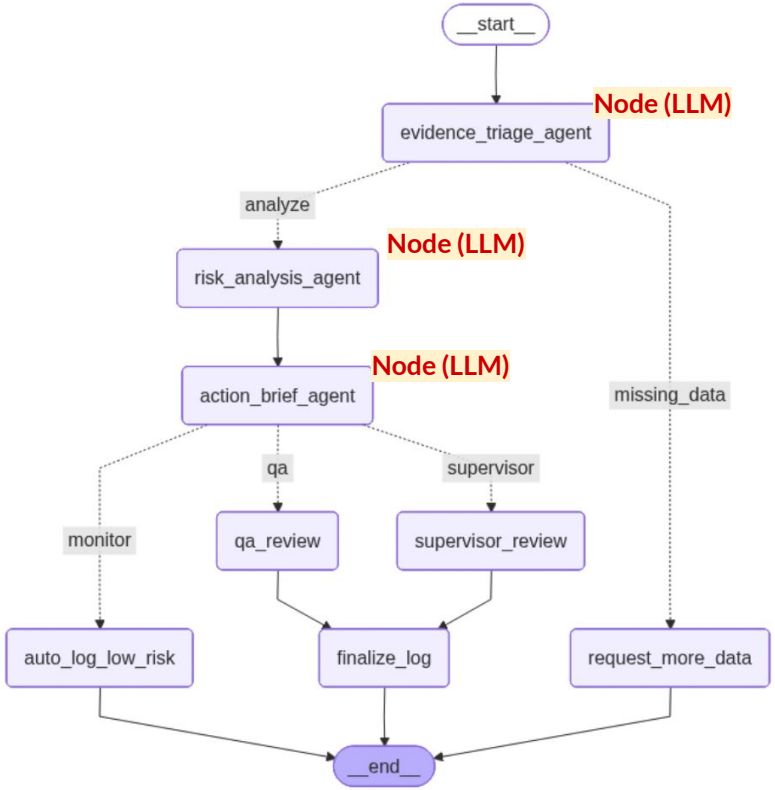
Concept 4: Conditional Edges

4. Conditional Edges (The Brain/Router)

- Unlike standard, fixed transitions, Conditional Edges allow the workflow to make dynamic routing decisions at runtime based on the evolving State.
- Instead of moving sequentially, a routing function evaluates the current state's properties (e.g., checking if an LLM recommended tool usage, or checking if a test passed) and routes the execution to one of multiple possible downstream nodes.

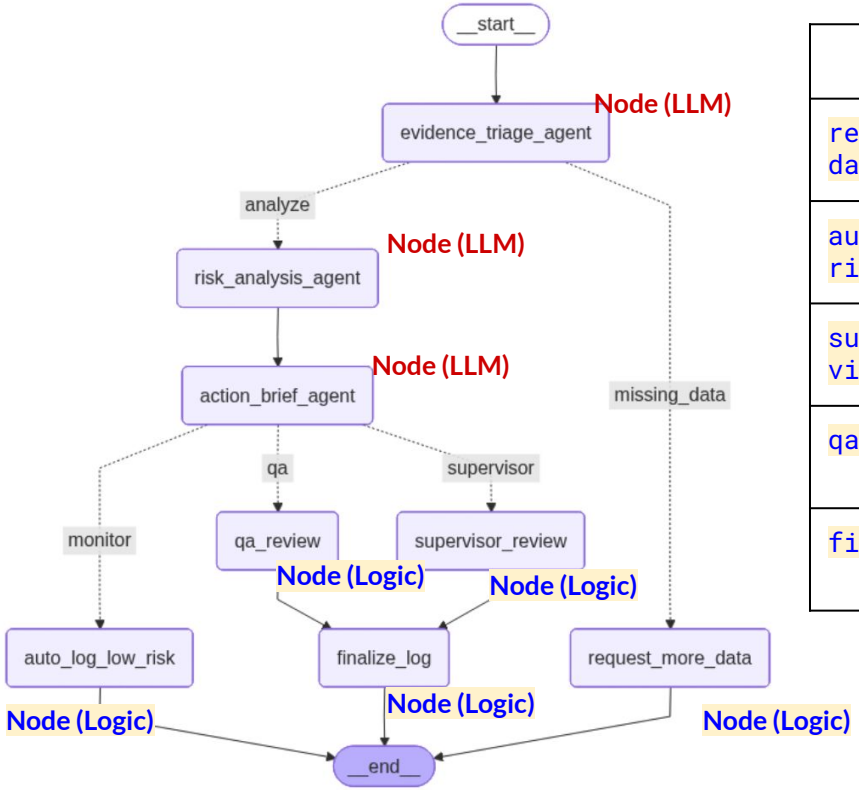


Our Demo : Agentic Workflow (1)



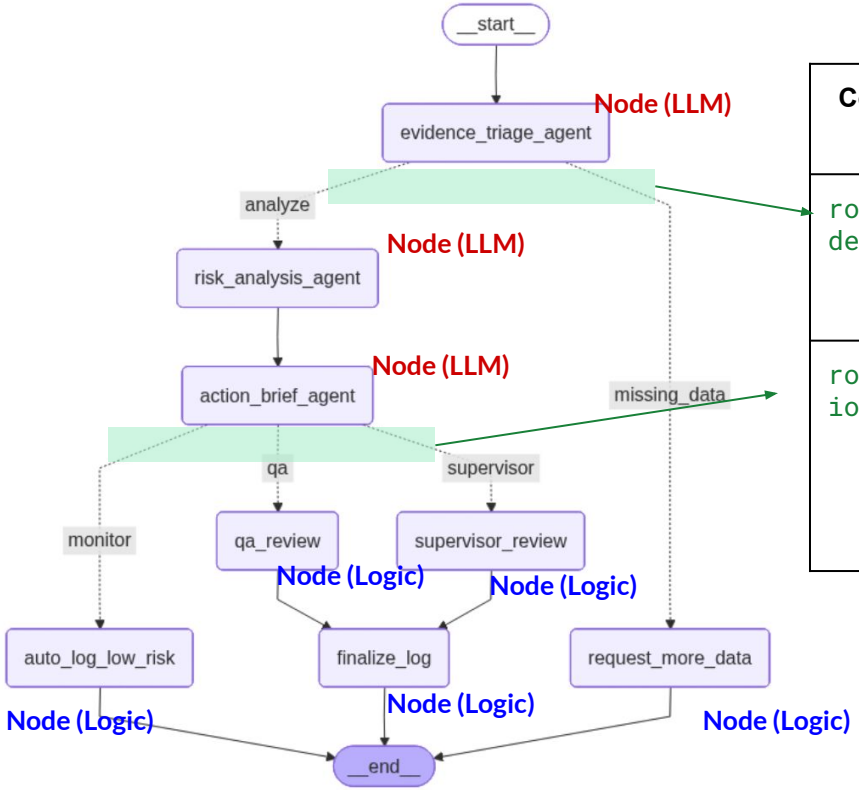
Node	Type	เหตุผล
`evidence_triage_agent`	**LLM Node + Logic Guardrail**	ใช้ LLM เช็ก evidence completeness แล้วมี hard rule ตรวจซ้ำ
`risk_analysis_agent`	**LLM Node**	ใช้ LLM วิเคราะห์ risk level: `LOW / MEDIUM / HIGH`
`action_brief_agent`	**LLM Node + Logic Guardrail**	ใช้ LLM เขียน action brief แต่ mapping recommendation ถูกบังคับด้วย rule

Our Demo : Agentic Workflow (2)



Node	Type	หน้าที่
`request_more_data`	**Logic Node**	ถ้าข้อมูลไม่ครบ ให้จบที่ waiting status
`auto_log_low_risk`	**Logic Node**	LOW risk → auto log
`supervisor_review`	**Human-in-the-loop Node / Logic Node**	MEDIUM risk → interrupt รอ supervisor decision
`qa_review`	**Human-in-the-loop Node / Logic Node**	HIGH risk → interrupt รอ QA/Food Safety decision
`finalize_log`	**Logic Node**	บันทึก human decision แล้วจบ workflow

Our Demo : Agentic Workflow (3)

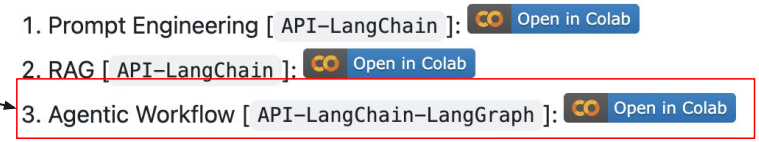


Conditional Edge Function	อยู่หลัง Node ไหน	Routing Logic
`route_after_evidence_triage`	หลัง `evidence_triage_agent`	ถ้า evidence ไม่ครบ → `request_more_data`, ถ้าครบ → `risk_analysis_agent`
`route_after_action_brief`	หลัง `action_brief_agent`	ถ้า `MONITOR` → `auto_log_low_risk`, ถ้า `INVESTIGATE` → `supervisor_review`, ถ้า `HOLD_FOR_QA_REVIEW` → `qa_review`

1 Node
ออกหลาย Edge = มี Conditional Edge

Our Demo : Agentic Workflow (4)

Run: `03_agentic_workflow.ipynb`



Observe: State updates, node execution, conditional routing, human review interrupt

Takeaway: Workflow = multiple connected steps with controllable routing.

Demo: Agentic Workflow (5)

0. Setup

Choose the provider in cell 5 before running the notebook.

1. Run `pip install langchain-openai` if you have an OpenAI API key.

Otherwise, uncomment and run `pip install langchain-groq`.

```
# %%capture
!pip install -qU langchain-openai
```

9

```
# if using groq api (Free Tier) => don't
# !pip install -qU langchain-groq
```

2. PROVIDER = …
   - “openai” if you have an OpenAI API Key
   - “groq” if you have a Groq API Key.

```
import os
import json
import re
import time
import getpass

PROVIDER = "groq"  # CHANGE THIS LINE ONLY: openai or groq

if PROVIDER == "openai":
    from langchain_openai import ChatOpenAI
    os.environ["OPENAI_API_KEY"] = getpass.getpass("Enter your OpenAI API key: ")
    default_llm = ChatOpenAI(
        model="gpt-5-nano",
        temperature=0.6,
        max_retries=2,
    )
elif PROVIDER == "groq":
    from langchain_groq import ChatGroq
    os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq API key: ")
    default_llm = ChatGroq(
        model="openai/gpt-oss-120b",
        temperature=0.6,
        max_tokens=None,
        timeout=None,
        max_retries=2,
    )
else:
```

Last
FAQ: Production

Prompt Engineering in Production

Prompt Engineering in Production

In production, prompts are managed as code, not copy-pasted.

Typical Flow:

App API → Prompt Template → LLM → Output Validator

What needs to be added?

- **Version Control:** Track prompt changes like code. (Sol: Git or LangSmith)
- **Guardrails:** Prevent Prompt Injection from external users. (Sol: at System Prompt / User Prompt)
- **Validation:** Ensure structured output formats Sol: Pydantic)

- **System Prompt (Define Rules & Teach Model)**
  - Hard Rules: Strict commands to ignore hidden user instructions.
  - Few-Shot Examples: Simulate injection cases; teach prevention/response.
- **User Prompt (Block Before Main Model)**
  - Rule-Based Check: Filter initial high-risk words/patterns.
  - LLM Input Scanner: Use an LLM to check user message intent (Safe/Unsafe) before processing.
- **Data Separation (Separate Data Context)**
  - API Format: Enforce strict variable structures (e.g., `role: "system"` and `role: "user"`).
  - This helps the model distinguish user data from system commands.

Multi-Brand Prompt Optimization

Different LLM brands require different prompt structures for optimal performance.

OpenAI (GPT)

Responds exceptionally well to **Role-playing** ("You are an expert...") and clear system message instructions.

Anthropic (Claude)

Optimized for **XML Tags** to separate instructions from context data.

Meta (Llama)

Highly sensitive to specific formatting tokens and structured **Instruction (Inst) tags**

Multi-Brand Prompt Optimization

OpenAI

[Prompt guidance | OpenAI API]

Example personality block for a steady task-focused assistant:

```
# Personality
You are a capable collaborator: approachable, steady, and direct. Assume the user is competent
and acting in good faith, and respond with patience, respect, and practical helpfulness.

Prefer making progress over stopping for clarification when the request is already clear enough
to attempt. Use context and reasonable assumptions to move forward. Ask for clarification only
when the missing information would materially change the answer or create meaningful risk, and
keep any question narrow.

Stay concise without becoming curt. Give enough context for the user to understand and trust the
answer, then stop. Use examples, comparisons, or simple analogies when they make the point
easier to grasp. When correcting the user or disagreeing, be candid but constructive. When an
error is pointed out, acknowledge it plainly and focus on fixing it.

Match the user's tone within professional bounds. Avoid emojis and profanity by default, unless
the user explicitly asks for that style or has clearly established it as appropriate for the
conversation.
```

Outcome-first prompts and stopping conditions

GPT-5.5 is strongest when the prompt defines the target outcome, success criteria, constraints, and available context, then lets the model choose the path.

For many tasks, describe the destination rather than every step. This gives the model room to choose the right search, tool, or reasoning strategy for the task.

Prefer this:

```
Resolve the customer's issue end to end.

Success means:
- the eligibility decision is made from the available policy and account data
- any allowed action is completed before responding
- the final answer includes completed_actions, customer_message, and blockers
- if evidence is missing, ask for the smallest missing field
```

Suggested prompt structure

Use this structure as a starting point for complex prompts. Keep each section short. Add detail only where it changes behavior.

```
Role: [1–2 sentences defining the model's function, context, and job]

# Personality
[tone, demeanor, and collaboration style]

# Goal
[user-visible outcome]

# Success criteria
[what must be true before the final answer]

# Constraints
[policy, safety, business, evidence, and side-effect limits]

# Output
[sections, length, and tone]

# Stop rules
[when to retry, fallback, abstain, ask, or stop]
```

Utilizing markdown tags (e.g., ###) and defining a personality or role for the model, along with clear constraints, enhances the performance of OpenAI models.

Multi-Brand Prompt Optimization

Claude

Prompting best practices - Claude API Docs

```
You are an AI physician's assistant. Your task is to help doctors diagnose possible patie

<documents>
  <document index="1">
    <source>patient_symptoms.txt</source>
    <document_content>
      {{PATIENT_SYMPTOMS}}
    </document_content>
  </document>
  <document index="2">
    <source>patient_records.txt</source>
    <document_content>
      {{PATIENT_RECORDS}}
    </document_content>
  </document>
  <document index="3">
    <source>patient01_appt_history.txt</source>
    <document_content>
      {{PATIENT01_APPOINTMENT_HISTORY}}
    </document_content>
  </document>
</documents>

Find quotes from the patient records and appointment history that are relevant to diagnos
```

Structure prompts with XML tags

XML tags help Claude parse complex prompts unambiguously, especially when your prompt mixes instructions, context, examples, and variable inputs. Wrapping each type of content in its own tag (e.g. `<instructions>`, `<context>`, `<input>`) reduces misinterpretation.

Best practices:

- Use consistent, descriptive tag names across your prompts.
- Nest tags when content has a natural hierarchy (documents inside `<documents>`, each inside `<document index="n">`).

Multi-Brand Prompt Optimization

[Llama 3.1 | Model Cards and Prompt formats]

Llama (Meta)
Example: Optimized for special tokens.

Special Tokens

Tokens	Description
`<|begin_of_text|>`	Specifies the start of the prompt.
`<|end_of_text|>`	Model will cease to generate more tokens. This token is generated only by the base models.
`<|finetune_right_pad_id|>`	This token is used for padding text sequences to the same length in a batch.
`<|start_header_id|>`	These tokens enclose the role for a particular message. The possible roles are: `[system, user, assistant, and ipython]`
`<|end_header_id|>`	
`<|eom_id|>`	End of message. A message represents a possible stopping point for execution where the model can inform the executor that a tool call needs to be made. This is used for multi-step interactions between the model and any available tools. This token is emitted by the model when the `Environment: ipython` instruction is used in the system prompt, or if the model calls for a built-in tool.
`<|eot_id|>`	End of turn. Represents when the model has determined that it has finished interacting with the user message that initiated its response. This is used in two scenarios: • at the end of a direct interaction between the model and the user • at the end of multiple interactions between the model and any available tools This token signals to the executor that the model has finished generating a response.
`<|python_tag|>`	Special tag used in the model's response to signify a tool call.

Multi-Brand Prompt Optimization

LLaMA

[Llama 3.1 | Model Cards and Prompt formats]

Step - 1 User Prompt & System prompts

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Environment: ipython
Tools: brave_search, wolfram_alpha
Cutting Knowledge Date: December 2023
Today Date: 23 July 2024

You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>

Can you help me solve this equation: x^3 - 4x^2 + 6x - 24 = 0<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Llama (Meta)
Example: Optimized for special tokens.

Built in Python based tool calling

The models are trained to use two built-in tools:

1. **Brave Search:** Tool call to perform web searches.
2. **Wolfram Alpha:** Tool call to perform complex mathematical calculations.

These can be turned on using the system prompt:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Environment: ipython
Tools: brave_search, wolfram_alpha
Cutting Knowledge Date: December 2023
Today Date: 23 July 2024

You are a helpful assistant<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is the current weather in Menlo Park, California?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

RAG in Production

Enterprise Vector Databases

Local tools like Chroma are great for demos, but production requires scaling, access control, and pipeline management.

Database	Core Characteristic	Best For
Pinecone	Serverless / Managed	Zero-Ops Scaling
Qdrant	Rust-based / High Speed	Complex Metadata Filtering
pgvector	PostgreSQL Extension	Existing SQL Infrastructure
Milvus	Distributed Architecture	Billion-Scale Enterprise

Understanding User Queries

Raw retrieval often fails on complex questions. We must process the query before searching.

Sub-Query Decomposition

Breaking a complex request (e.g., "Compare X and Y") into smaller, individual queries before combining the answers.

Query Expansion

Generating synonyms and variations of the user's prompt to ensure no relevant documents are missed.

Query Routing

Classifying what the user wants to route them to the right tool (e.g., Vector Search vs. SQL Database).

Thank You